

Master Project Specification Document (MPSD)

Project: ISF Playground - Combined Sprint 3 & Sprint 4 **Version:** 2.0 (Client Version) **Date:** October 17, 2025
Sprint Duration: 28 Days (4 Weeks)

1. MPSD Introduction & Combined Sprint Overview

1.1. Purpose of This Master Project Specification Document

This Master Project Specification Document (MPSD) serves as the exhaustive, unambiguous, and universally agreed-upon blueprint for the combined execution of ISF Playground Sprint 3 and Sprint 4. It is the single source of truth, meticulously detailing every facet of the features planned for this combined 28-day development effort. Its primary purpose is to ensure complete alignment among all stakeholders—including the Client, Project Manager, Design Team, and Development Team—before the sprint development work commences. This MPSD will be the definitive reference for scope management, enabling the Project Manager to clearly delineate agreed-upon features from out-of-scope requests, thereby ensuring focused and efficient development across both sprint deliverables.

1.2. Project Overview: ISF Playground - Combined Sprint 3 & 4

This combined sprint represents a strategic consolidation of the ISF Playground development roadmap, executing Sprint 3 and Sprint 4 features together over a 28-day period. Sprint 3 focuses on mobile application development for Coaches, Admins, and Balagruh In-Charges, enabling attendance tracking and media management on the go. Sprint 4 completes the safety and communication ecosystem with emergency SOS functionality and internal messaging capabilities. This combined execution approach leverages improved development processes and system architecture, allowing parallel development of desktop emergency features and mobile applications while maintaining system cohesion. The entire system continues to be built on the Electron.js desktop application foundation with a Node.js backend and MongoDB database, now extending to React Native mobile platforms.

1.3. Combined Sprint Goals & Objectives

The primary goals for this unified 28-day sprint are:

Sprint 3 Core Objectives:

- 1. Launch Mobile Application:** Develop React Native application for iOS and Android platforms, enabling staff to access key features on mobile devices.
- 2. Enable Mobile Attendance:** Implement facial recognition-based attendance system accessible from mobile devices, streamlining daily attendance marking.
- 3. Provide Mobile Media Management:** Allow Coaches to upload course content (videos, documents) directly from mobile devices.
- 4. Deliver Mobile Analytics:** Provide performance dashboards and reporting capabilities accessible on mobile platforms.
- 5. Establish Push Notifications:** Implement Firebase Cloud Messaging for real-time alerts and updates.

Sprint 4 Core Objectives:

1. **Implement SOS Emergency System:** Create desktop-based emergency alert system for students with mobile alert reception for staff.
2. **Build Multi-Tier Escalation:** Develop intelligent escalation workflow ensuring emergency responses within defined timeframes.
3. **Enable Internal Messaging:** Implement WhatsApp-like messaging system for staff coordination and communication.
4. **Integrate WhatsApp Notifications:** Extend communication capabilities with WhatsApp Business API for external notifications.
5. **Track Student Health:** Create health monitoring system with trend analysis and SOS correlation.

1.4. Development Strategy

To achieve the 28-day timeline, the development will be organized sequentially with early integration:

Sequential with Early Integration (Recommended):

Weeks 1-2: Mobile Foundation (Sprint 3 Core)

- Week 1: Core mobile app, authentication, navigation
- Week 2: Attendance tracking, media upload, reporting

Weeks 2.5: Integration Testing Checkpoint

- End-to-end mobile authentication
- Notification delivery verification
- WebSocket connection stability

Week 3: Sprint 3 Features + Sprint 4 Foundation

- Sprint 3: Mobile notifications, analytics dashboard
- Sprint 4: Desktop SOS trigger, mobile alert routing

Week 4: Sprint 4 Features + Integration

- Internal messaging module
- Health tracking system
- WhatsApp integration
- Full system integration testing
- Production readiness verification

Cross-Sprint Dependencies:

- Shared push notification infrastructure (FCM)
- Unified WebSocket real-time layer
- Common AWS S3 media storage
- Integrated authentication system

2. Target Users/Personas for Combined Sprint Features

Primary Personas:

- **Coach Rajesh (Age 32, Sports Coach):** The primary mobile user who will use the mobile app to upload training videos from his phone, check student attendance remotely, view performance reports during his commute, and respond to emergency SOS alerts immediately. Rajesh experiences the complete mobile-first workflow enabled by Sprint 3 and receives critical emergency notifications through Sprint 4.
- **Admin Priya (Age 28, System Administrator):** The system orchestrator with expanded mobile capabilities. Manages the platform from both desktop and mobile, accessing comprehensive dashboards on her iPhone, viewing attendance rates across multiple Balagruhs, and serving as Tier 2 escalation contact for SOS alerts. She coordinates emergency response through the messaging system.
- **Balagruh In-Charge Sunita (Age 45, Facility Manager):** The daily operations manager who will use her budget Android phone to mark attendance with photos, track student health records, receive attendance alerts, and enter routine health checkups. When students' vitals are abnormal, she receives immediate alerts and can link health incidents to SOS alerts.

Secondary Personas:

- **Student Arjun (Age 16):** The desktop app user who accesses the prominent SOS button when feeling unwell. Within 5 seconds, Coach Rajesh receives an alert on his mobile phone, acknowledges, and arrives within 3 minutes. Arjun's health data is correlated with SOS incidents for pattern analysis.
- **Parent/Guardian:** External user receiving WhatsApp notifications for daily attendance summaries and immediate alerts if their child triggers an SOS emergency.

3. High-Level Combined Sprint Scope

3.1. What's In Scope for the 28-Day Combined Sprint

Sprint 3 - Mobile App Development Features:

Mobile Application Foundation:

- React Native mobile app for iOS (15.1+) and Android (8.0+)
- JWT token-based authentication with biometric support (Face ID, Touch ID, Fingerprint)
- Role-based navigation and dashboards
- Offline-first architecture with queue & sync capabilities
- App store deployment preparation (TestFlight, Firebase App Distribution)

Mobile Attendance Tracking:

- Camera/gallery integration for photo capture
- Photo upload to AWS S3 with automatic compression (5MB → ~2MB)
- Integration with facial recognition backend (existing system)
- Real-time attendance processing and results display
- Manual override and verification interface
- Attendance history and reporting
- Offline photo queuing with automatic sync when online

Mobile Media Management:

- Multi-file upload (videos up to 500MB, documents up to 50MB, images up to 25MB)
- Course/module association workflow
- Batch upload with progress tracking
- Content publishing workflow
- Preview and metadata editing
- S3 presigned URL generation for secure uploads

Mobile Analytics & Reporting:

- Performance dashboard with key metrics (attendance rates, course completion, coin distribution)
- Date range filtering and drill-down capabilities
- Visual charts using React Native Chart libraries
- Top performers lists and leaderboards
- Export capabilities (PDF/CSV generation)
- Real-time data synchronization

Push Notification Infrastructure:

- Firebase Cloud Messaging (FCM) integration
- Device token registration and management
- Background and foreground notification handling
- Deep linking to relevant screens (course, attendance, SOS)
- Notification preferences and quiet hours
- Notification history screen with mark-as-read

Sprint 4 - Emergency & Communication Features:

SOS Emergency System (Desktop → Mobile):

- Desktop SOS trigger button with prominent placement on student interface
- Emergency category selection (Medical, Safety, Mental Health, Other)
- Mobile alert reception with high-priority push notifications
- Real-time status tracking (sent → acknowledged → responding → resolved)
- Multi-tier escalation workflow:
 - Tier 1: Balagruh Coaches + In-Charge (0-2 minutes)
 - Tier 2: Admins + SMS/WhatsApp (2-5 minutes)
 - Tier 3: Broadcast to all staff (5+ minutes)
- Response coordination dashboard showing who's responding
- Compliance and audit logging for all SOS events

Internal Messaging Module:

- 1-on-1 direct messaging between staff members
- Group conversations for teams and Balagruhs
- Text, image, and file attachments (25MB images, 100MB videos, 25MB documents)
- Read receipts and typing indicators
- Message search and conversation history
- Real-time delivery via WebSocket with offline queuing
- Unread count badges and notification integration

WhatsApp Business API Integration:

- Template message support for standardized communications
- SOS alert notifications to guardians
- Daily attendance summaries for parents
- Opt-in/opt-out management for recipients
- Delivery status tracking and retry logic
- Rate limiting compliance (per WhatsApp Business API terms)

Student Health Tracking:

- Health metrics entry interface (weight, height, temperature, blood pressure, heart rate)
- Symptom checklist (fever, cough, headache, fatigue, etc.)
- Health document uploads (medical reports, prescriptions)
- Trend analysis and growth charts visualization
- Abnormal value alerts:
 - Temperature > 37.5°C warning, > 39°C critical
 - Weight change > 5% warning
- SOS incident correlation for pattern detection
- Health report exports for medical professionals

Shared Infrastructure (Cross-Sprint):**WebSocket Real-Time Layer:**

- Socket.io server implementation with JWT authentication
- Room-based messaging (user rooms, balagruh rooms, conversation rooms)
- Presence tracking (online/offline status)
- Message queuing for offline users
- Automatic reconnection handling with exponential backoff

AWS S3 Media Storage:

- Presigned URL generation for secure direct uploads
- Folder organization (attendance-photos/, course-content/, health-documents/)
- Image optimization pipeline (automatic compression, thumbnail generation)
- CDN integration with CloudFront for faster delivery
- Backup and versioning policies (30-day retention)

3.2. What's Out of Scope for the Combined Sprint**Sprint 3 Exclusions:**

- Student mobile application (Students continue using desktop app only)
- Parent/Guardian mobile application (Parents use WhatsApp only)
- Video conferencing or live streaming features
- Advanced analytics with machine learning predictions
- Integration with external Learning Management Systems
- Multi-language support for mobile UI (English only, content translation exists)

Sprint 4 Exclusions:

- Direct emergency services integration (911/108 calling)
- Telemedicine or doctor video consultation features
- Prescription management or pharmacy integration
- Integration with wearable health devices (Fitbit, Apple Watch)
- Automated medical diagnosis or AI-powered health recommendations
- Voice/video calling within messaging (text and files only)
- Broadcast channels for announcements (separate from Mann ki Baat)

Deferred to Future Sprints:

- Mobile app for Students (Future sprint consideration)
- Advanced health analytics with ML
- Calendar and event management system
- Task assignment and tracking beyond SOS follow-up
- Performance review and appraisal system
- Advanced reporting with predictive analytics

4. Target Audience for this MPSD

- **Primary:** Project Manager (for sprint coordination, resource allocation, client communication, and integrated scope management)
- **Secondary:**
 - **Development Team (Frontend & Backend):** For precise implementation details across both mobile and emergency features
 - **QA Team:** For comprehensive test planning covering desktop-to-mobile integration scenarios
 - **Design Team:** For ensuring UI/UX consistency across mobile and desktop experiences
 - **Client (ISF):** For final approval and understanding of the 28-day delivery timeline

5. Document Conventions

- **UI Elements:** Referred to by their visible label text in "quotation marks" or by descriptive name with suggested IDs (e.g., btn-mark-attendance, btn-trigger-sos)
- **User Roles:** Capitalized consistently (Student, Coach, Admin, Balagruh In-Charge, Amma)
- **Placeholders:** Dynamic data shown in [square_brackets] (e.g., [StudentName], [SOSCategory], [CoinBalance])
- **API Endpoints:** Represented as METHOD /path/to/endpoint with versioning where applicable
- **Sprint Attribution:** Features are tagged with [S3] for Sprint 3, [S4] for Sprint 4, or [SHARED] when impacting both sprints
- **Priority Levels:** P0 (Critical - Must have), P1 (High Priority), P2 (Medium Priority), P3 (Low Priority)

6. References to Source Documents

This combined MPSD synthesizes and integrates information from:

- Playground Platform - 5 Sprint Plan (16 February 2025)
 - ISF Sprint 3 - Overview and Scope
 - ISF Sprint 4 - Overview and Scope
 - Sprint 2+5 Combined MPSD v3.0 (structure template)
 - Client-provided mobile UI mockups (definitive source for mobile implementation)
 - Completed Sprint 1, Sprint 2, and Sprint 5 implementations
 - User feedback and decisions from Sprint planning sessions
-

7. Global Elements & Standards (Combined Sprint Context)

7.1. Branding Guidelines

- **Consistency Requirement:** Mobile app must maintain identical branding to desktop application
- **Color Palette:**
 - Primary: #1E40AF (Blue-700) - Buttons, headers, active states
 - Secondary: #10B981 (Green-500) - Success messages, confirmations
 - Accent: #F59E0B (Amber-500) - Warnings, highlights
 - Error/Alert: #EF4444 (Red-500) - Errors, validations
 - SOS Emergency: #DC2626 (Red-600) - SOS button, urgent alerts
- **Typography:**
 - Headings: Inter Bold
 - Body: Inter Regular
 - Buttons: Inter Semi-Bold
- **Logo Placement:** ISF Playground logo consistent in all module headers (both mobile and desktop)

7.2. Responsive Design & Performance

- **Mobile Target Resolutions:**
 - iPhone SE: 375x667 (small phone baseline)
 - iPhone 12: 390x844 (standard iPhone)
 - Samsung Galaxy: 360x800 (standard Android)
 - iPad: 768x1024 (tablet support)
- **Performance Requirements:**
 - Mobile app cold start: < 3 seconds
 - Screen transitions: < 300ms
 - API response time: < 500ms (p95)
 - Image loading: Progressive (placeholder → full)
 - SOS alert delivery: < 5 seconds end-to-end
 - Mobile bundle size: < 25MB (optimized with code splitting)
- **Desktop Target Resolution:** 1366x768 (maintained from previous sprints)

7.3. Accessibility Standards

- **WCAG 2.1 Level AA:** Applied consistently across mobile and desktop
- **Touch Targets:** Minimum 44x44px (iOS HIG), recommended 48x48px (Material Design)

- **Keyboard Navigation:** Full support on desktop features
- **Screen Reader Compatibility:**
 - VoiceOver (iOS) support for mobile
 - TalkBack (Android) support for mobile
 - NVDA/JAWS support for desktop
- **Color Contrast:** Minimum 4.5:1 for normal text, 3:1 for large text
- **Dynamic Type:** Font scaling support on mobile platforms

7.4. Unified Navigation Structure

- **Desktop App (Electron) - Enhanced:**
 - Student Dashboard
 - [Existing] Home, Courses, Attendance, Shop, Wallet, Profile
 - [S4] **[SOS Button]** ← Prominent, always visible, red color
 - Coach/Admin Dashboard
 - [Existing] Home, Students, Courses, Attendance, Shop Management, Reports
 - [S4] **SOS Alerts** badge count (high priority notifications)
- **Mobile App (React Native) - New:**
 - Bottom Tab Navigation
 - [S3]  Dashboard (role-specific quick actions)
 - [S3]  Attendance (Balagruh In-Charge only)
 - [S3]  Media (Coach/Admin only)
 - [S4]  Messages (all staff roles)
 - [S3]  Profile (settings, preferences)
 - Dashboard Quick Actions (Role-based)
 - [S3] Upload Course Content (Coach/Admin)
 - [S3] Mark Attendance (Balagruh In-Charge)
 - [S3] View Reports (Admin/Coach)
 - [S4] SOS Alerts (Coach/Admin) - Badge count
 - [S4] Health Records (Balagruh In-Charge)
 - [SHARED] Notifications (all roles)
- **Notification Center (Unified - Both Platforms):**
 - Notification Bell Icon with Badge Count
 - [S3] Attendance completed/failed
 - [S4] SOS Alert (urgent priority - red)
 - [S4] New message (high priority)
 - [S3] Low inventory alert (medium priority)
 - [S3] Daily report ready (low priority)

Detailed Feature & Module Breakdown

Section A: Sprint 3 - Mobile App Development

8. Mobile App Foundation & Core Infrastructure [S3]

- **Feature ID:** S3-F01
 - **Feature Name:** Mobile App Foundation & Core Infrastructure
 - **Module:** Mobile Core
 - **Priority:** P0 (Critical Path)
 - **Development Timeline:** Week 1-2 (10 days)
 - **Dependencies:** None (Foundation)
-

Story S3-F01-STORY-01: Mobile App Initialization & Project Setup

User Story:

As a **Developer**, I want to set up the mobile app project with React Native and TypeScript, so that we have a solid foundation for mobile development with proper architecture and tooling.

Acceptance Criteria:

1. Project Initialization:

- ☐ React Native project created with TypeScript support (v0.72.6)
- ☐ Project runs successfully on iOS simulator (Xcode 16.1+)
- ☐ Project runs successfully on Android emulator (Android Studio)
- ☐ Environment configuration files (.env.development, .env.staging, .env.production)
- ☐ Git repository configured with proper .gitignore

2. Project Structure:

- ☐ Modular folder structure implemented (screens, components, services, store, utils)
- ☐ Follows Sprint 5 patterns (module isolation, separation of concerns)
- ☐ ESLint and Prettier configured for code consistency
- ☐ TypeScript strict mode enabled with proper tsconfig.json

3. Navigation Framework:

- ☐ React Navigation v6 installed and configured
- ☐ Stack navigator for authentication flow
- ☐ Bottom tab navigator for main app sections
- ☐ Drawer navigator for additional menu items (settings, help)
- ☐ Deep linking configuration for notifications (opens specific screens)

4. State Management:

- ☐ Zustand installed and configured (consistent with web app patterns)
- ☐ Global stores created: AuthStore, NotificationStore, ConfigStore
- ☐ Persistent storage configured (AsyncStorage for data persistence)
- ☐ Hydration logic for offline-first capabilities

5. UI Component Library:

- ☐ Base component library selected (React Native Paper or custom components)
- ☐ Theme provider configured with ISF branding colors
- ☐ Reusable components: Button, TextInput, Card, Modal, Badge
- ☐ Loading indicators (spinner, skeleton screens) and error states

6. Build Configuration:

- ☐ iOS build configuration (Xcode project settings, Info.plist)
- ☐ Android build configuration (Gradle files, AndroidManifest.xml)
- ☐ App icon and splash screen assets added (1024x1024 source)
- ☐ App name and bundle identifier configured (com.isfplayground.mobile)
- ☐ Development, staging, and production build variants

7. Development Tools:

- ☐ React Native Debugger integration
- ☐ Flipper configured for debugging (network inspector, logs)
- ☐ Hot reload working correctly
- ☐ Build scripts in package.json (ios, android, start, test)

Technical Implementation Details:

Technology Stack:

```
{
  "dependencies": {
    "react": "18.2.0",
    "react-native": "0.72.6",
    "@react-navigation/native": "^6.1.9",
    "@react-navigation/stack": "^6.3.20",
    "@react-navigation/bottom-tabs": "^6.5.11",
    "zustand": "^4.4.6",
    "@react-native-async-storage/async-storage": "^1.19.5",
    "axios": "^1.6.2",
    "react-native-config": "^1.5.1",
    "@react-native-firebase/app": "^18.6.1",
    "@react-native-firebase/messaging": "^18.6.1"
  },
  "devDependencies": {
    "@typescript-eslint/eslint-plugin": "^6.12.0",
    "@typescript-eslint/parser": "^6.12.0",
    "eslint": "^8.54.0",
    "prettier": "^3.1.0",
    "typescript": "^5.3.2",
    "@types/react": "^18.2.0",
    "@types/react-native": "^0.72.0"
  }
}
```

Project Structure:

```

mobile/
├── src/
│   ├── screens/
│   │   ├── auth/
│   │   │   ├── LoginScreen.tsx
│   │   │   └── BiometricSetupScreen.tsx
│   │   ├── dashboard/
│   │   │   └── DashboardScreen.tsx
│   │   ├── attendance/
│   │   │   ├── AttendancePhotoScreen.tsx
│   │   │   └── AttendanceResultsScreen.tsx
│   │   ├── media/
│   │   │   └── MediaUploadScreen.tsx
│   │   ├── messaging/
│   │   │   ├── ConversationListScreen.tsx
│   │   │   └── ChatScreen.tsx
│   │   ├── sos/
│   │   │   └── SOSAlertScreen.tsx
│   │   ├── profile/
│   │   │   └── ProfileScreen.tsx
│   │   └── settings/
│   │       └── SettingsScreen.tsx
│   ├── components/
│   │   ├── common/
│   │   │   ├── Button.tsx
│   │   │   ├── TextInput.tsx
│   │   │   ├── Card.tsx
│   │   │   ├── Modal.tsx
│   │   │   └── Badge.tsx
│   │   └── navigation/
│   │       ├── BottomTabNavigator.tsx
│   │       └── DrawerNavigator.tsx
│   ├── services/
│   │   ├── api/
│   │   │   ├── apiClient.ts
│   │   │   ├── authApi.ts
│   │   │   ├── attendanceApi.ts
│   │   │   ├── mediaApi.ts
│   │   │   ├── messagingApi.ts
│   │   │   └── sosApi.ts
│   │   ├── storage/
│   │   │   └── secureStorage.ts
│   │   └── notifications/
│   │       └── fcmService.ts
│   ├── store/
│   │   ├── authStore.ts
│   │   ├── notificationStore.ts
│   │   └── configStore.ts
│   ├── utils/
│   │   ├── validation.ts
│   │   ├── formatting.ts
│   │   └── permissions.ts
│   └── constants/

```

```
├── colors.ts
├── sizes.ts
├── config.ts
├── types/
│   ├── user.ts
│   ├── attendance.ts
│   ├── notification.ts
│   ├── message.ts
│   └── sos.ts
├── navigation/
│   └── RootNavigator.tsx
├── App.tsx
├── android/
├── ios/
├── .env.development
├── .env.staging
├── .env.production
├── package.json
├── tsconfig.json
└── README.md
```

Environment Configuration:

```
# .env.development
REACT_APP_API_BASE_URL=http://localhost:5000/api
REACT_APP_WS_URL=ws://localhost:5000
REACT_APP_S3_BUCKET_URL=https://isf-playground-dev.s3.amazonaws.com

# .env.production
REACT_APP_API_BASE_URL=https://api.isfplayground.org/api
REACT_APP_WS_URL=wss://api.isfplayground.org
REACT_APP_S3_BUCKET_URL=https://isf-playground-prod.s3.amazonaws.com
```

API Client Setup:

```
// src/services/api/apiClient.ts
import axios, { AxiosInstance } from 'axios';
import Config from 'react-native-config';
import { getToken, clearTokens } from '../storage/secureStorage';

const apiClient: AxiosInstance = axios.create({
  baseURL: Config.REACT_APP_API_BASE_URL,
  timeout: 30000,
  headers: {
    'Content-Type': 'application/json',
  },
});

// Request interceptor to add auth token
```

```
apiClient.interceptors.request.use(
  async (config) => {
    const token = await getToken();
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

// Response interceptor for error handling
apiClient.interceptors.response.use(
  (response) => response,
  async (error) => {
    if (error.response?.status === 401) {
      // Token expired, clear tokens and redirect to login
      await clearTokens();
      // Navigation logic handled by NavigationService
    }
    return Promise.reject(error);
  }
);

export default apiClient;
```

Frontend Components:

- `LoginScreen.tsx`: Authentication UI with username/password inputs
- `DashboardScreen.tsx`: Main dashboard with role-specific quick actions
- `BottomTabNavigator.tsx`: Bottom tab navigation bar
- `Button.tsx`, `TextInput.tsx`: Reusable UI components

API Endpoints:

- None (Foundation setup only, authentication endpoints in next story)

Data Models:

- None (Foundation setup only, schemas in subsequent stories)

UI/UX Wireframe Description:

App Launch Flow:

1. Splash screen with ISF logo (2 seconds, animated fade-in)
2. Check for stored auth token in secure storage
3. If token exists and valid → Navigate to Dashboard
4. If no token or invalid → Navigate to Login Screen

Navigation Structure:

```

Root Navigator (Stack)
├── Auth Stack
│   └── Login Screen
└── Main Stack
    ├── Bottom Tab Navigator
    │   ├── Tab 1: Dashboard
    │   ├── Tab 2: Attendance (role-specific visibility)
    │   ├── Tab 3: Media (role-specific visibility)
    │   ├── Tab 4: Messages
    │   └── Tab 5: Profile
    └── Modal Screens (overlays)
        ├── SOS Alert Modal (high priority)
        ├── Notification Detail
        └── Settings
    
```

Testing Checklist:

- ☐ App builds successfully on iOS (Xcode 16.1+)
- ☐ App builds successfully on Android (API 26+)
- ☐ Navigation between screens works smoothly (< 300ms transitions)
- ☐ Hot reload functions correctly during development
- ☐ Environment variables load properly from .env files
- ☐ TypeScript compilation has no errors
- ☐ ESLint shows no warnings

Story S3-F01-STORY-02: Mobile Authentication System

User Story:

As a **Coach/Admin/Balagruh In-Charge**, I want to log into the mobile app using my existing credentials and optionally use biometric authentication, so that I can securely access mobile features quickly.

Acceptance Criteria:

1. Login Screen:

- ☐ Clean, professional login UI with ISF branding
- ☐ Username text field with appropriate keyboard type (email/default)
- ☐ Password text field with secure entry and show/hide toggle (eye icon)
- ☐ [Log In] button (disabled until both fields filled, loading spinner when active)
- ☐ Loading indicator during authentication (spinner with "Signing you in...")
- ☐ Error messages displayed clearly below form (red background, white text)
- ☐ "Forgot Password?" link (links to desktop app instruction text)

2. Authentication Flow:

- ☐ JWT token-based authentication (reuses existing backend `/api/auth/login`)
- ☐ Successful login stores JWT token securely (iOS Keychain, Android Keystore)

- ☐ Successful login stores refresh token (30-day expiration)
- ☐ User object stored in AuthStore (Zustand with persistence)
- ☐ Navigate to Dashboard on successful authentication

3. Token Management:

- ☐ JWT token included in all API requests (Authorization: Bearer header)
- ☐ Token refresh mechanism (automatic when token expires during API call)
- ☐ Refresh token rotation (security best practice, new token on each refresh)
- ☐ Auto-login if valid token exists on app launch
- ☐ Logout clears all tokens and user data from storage

4. Biometric Authentication:

- ☐ After first successful login, prompt to enable biometric auth
- ☐ Support Face ID (iOS), Touch ID (iOS), Fingerprint (Android)
- ☐ Biometric auth bypasses username/password entry (direct to dashboard)
- ☐ Fallback to username/password if biometric fails (3 attempts)
- ☐ Option to disable biometric auth in settings
- ☐ Re-prompt for credentials every 30 days (security requirement)

5. Session Management:

- ☐ Session timeout after 24 hours of inactivity
- ☐ Session timeout warning 5 minutes before expiry
- ☐ Ability to extend session with user action ("Stay logged in" button)
- ☐ Logout functionality (clears tokens, redirects to login)
- ☐ Force logout if token is revoked (backend notification)

6. Role-Based Access:

- ☐ User role stored in AuthStore (Coach, Admin, Balagruh In-Charge)
- ☐ Dashboard shows role-appropriate quick actions
- ☐ Navigation tabs filtered by role permissions
- ☐ API requests include role in token claims (verified server-side)

7. Error Handling:

- ☐ Invalid credentials: "Incorrect username or password. Please try again."
- ☐ Network error: "Unable to connect. Please check your internet connection."
- ☐ Server error: "Something went wrong. Please try again later."
- ☐ Account locked: "Your account is temporarily locked. Contact admin."
- ☐ Biometric failure: Fallback to password entry after 3 failed attempts

Technical Implementation Details:

Authentication Store (Zustand):

```
// src/store/authStore.ts
import { create } from 'zustand';
```

```
import { persist } from 'zustand/middleware';
import AsyncStorage from '@react-native-async-storage/async-storage';
import { User } from '../types/user';

interface AuthState {
  user: User | null;
  token: string | null;
  refreshToken: string | null;
  isAuthenticated: boolean;
  biometricEnabled: boolean;
  login: (username: string, password: string) => Promise<void>;
  logout: () => Promise<void>;
  refreshTokenFn: () => Promise<void>;
  enableBiometric: () => Promise<void>;
  disableBiometric: () => void;
}

export const useAuthStore = create<AuthState>()(
  persist(
    (set, get) => ({
      user: null,
      token: null,
      refreshToken: null,
      isAuthenticated: false,
      biometricEnabled: false,

      login: async (username: string, password: string) => {
        const response = await authApi.login(username, password);
        const { token, refreshToken, user } = response.data;

        // Store tokens securely in Keychain/Keystore
        await secureStorage.setToken(token);
        await secureStorage.setRefreshToken(refreshToken);

        set({
          user,
          token,
          refreshToken,
          isAuthenticated: true,
        });
      },

      logout: async () => {
        const { token } = get();
        await authApi.logout(token);
        await secureStorage.clearTokens();

        set({
          user: null,
          token: null,
          refreshToken: null,
          isAuthenticated: false,
          biometricEnabled: false,
        });
      },
    })
  )
);
```



```

    },

    refreshTokenFn: async () => {
      const { refreshToken } = get();
      if (!refreshToken) throw new Error('No refresh token');

      const response = await authApi.refreshToken(refreshToken);
      const { token: newToken } = response.data;

      await secureStorage.setToken(newToken);
      set({ token: newToken });
    },

    enableBiometric: async () => {
      const biometricType = await biometricAuth.getSupportedType();
      if (biometricType) {
        set({ biometricEnabled: true });
      }
    },

    disableBiometric: () => {
      set({ biometricEnabled: false });
    },
  )),
  {
    name: 'auth-storage',
    storage: AsyncStorage,
    partialize: (state) => ({
      user: state.user,
      biometricEnabled: state.biometricEnabled,
    }),
  }
)
);

```

User Type:

```

// src/types/user.ts
export interface User {
  _id: string;
  username: string;
  role: 'Coach' | 'Admin' | 'Balagruh In-Charge';
  balagruhId?: string;
  balagruhName?: string;
  permissions: string[];
  profilePicture?: string;
  email?: string;
  phone?: string;
}

```

Secure Storage Service:

```
// src/services/storage/secureStorage.ts
import * as Keychain from 'react-native-keychain';

const TOKEN_KEY = 'isf_auth_token';
const REFRESH_TOKEN_KEY = 'isf_refresh_token';

export const secureStorage = {
  async setToken(token: string): Promise<void> {
    await Keychain.setGenericPassword(TOKEN_KEY, token, {
      service: 'com.isfplayground.auth',
    });
  },

  async getToken(): Promise<string | null> {
    const credentials = await Keychain.getGenericPassword({
      service: 'com.isfplayground.auth',
    });
    return credentials ? credentials.password : null;
  },

  async setRefreshToken(token: string): Promise<void> {
    await Keychain.setGenericPassword(REFRESH_TOKEN_KEY, token, {
      service: 'com.isfplayground.refresh',
    });
  },

  async getRefreshToken(): Promise<string | null> {
    const credentials = await Keychain.getGenericPassword({
      service: 'com.isfplayground.refresh',
    });
    return credentials ? credentials.password : null;
  },

  async clearTokens(): Promise<void> {
    await Keychain.resetGenericPassword({ service: 'com.isfplayground.auth' });
    await Keychain.resetGenericPassword({ service: 'com.isfplayground.refresh' });
  },
};
```

Biometric Authentication Service:

```
// src/services/auth/biometricAuth.ts
import ReactNativeBiometrics, { BiometryTypes } from 'react-native-biometrics';

const rnBiometrics = new ReactNativeBiometrics();

export const biometricAuth = {
  async getSupportedType(): Promise<BiometryTypes | null> {
```

```

    const { biometryType } = await rnBiometrics.isSensorAvailable();
    return biometryType;
  },

  async authenticate(): Promise<boolean> {
    const { success } = await rnBiometrics.simplePrompt({
      promptMessage: 'Authenticate to access ISF Playground',
      cancelButtonText: 'Use Password',
    });
    return success;
  },

  async isAvailable(): Promise<boolean> {
    const { available } = await rnBiometrics.isSensorAvailable();
    return available;
  },
};

```

Frontend Components:

LoginScreen.tsx:

```

// src/screens/auth/LoginScreen.tsx
import React, { useState, useEffect } from 'react';
import { View, Text, StyleSheet, Alert } from 'react-native';
import { useAuthStore } from '../../../store/authStore';
import { biometricAuth } from '../../../services/auth/biometricAuth';
import Button from '../../../components/common/Button';
import TextInput from '../../../components/common/TextInput';

const LoginScreen: React.FC = ({ navigation }) => {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  const { login, biometricEnabled } = useAuthStore();

  useEffect(() => {
    // Attempt biometric login if enabled
    if (biometricEnabled) {
      attemptBiometricLogin();
    }
  }, []);

  const attemptBiometricLogin = async () => {
    const isAvailable = await biometricAuth.isAvailable();
    if (!isAvailable) return;

    const success = await biometricAuth.authenticate();
    if (success) {
      // Biometric success, proceed with auto-login (assumes token still valid)
    }
  };

```

```
        navigation.replace('Main');
    }
};

const handleLogin = async () => {
    setLoading(true);
    setError('');

    try {
        await login(username, password);
        navigation.replace('Main');
    } catch (err: any) {
        if (err.response?.status === 401) {
            setError('Incorrect username or password. Please try again.');
```

```
        } else if (err.message === 'Network Error') {
            setError('Unable to connect. Please check your internet connection.');
```

```
        } else {
            setError('Something went wrong. Please try again later.');
```

```
        }
    } finally {
        setLoading(false);
    }
};

return (
    <View style={styles.container}>
        <Text style={styles.title}>ISF Playground</Text>
        <Text style={styles.subtitle}>Mobile App</Text>

        <TextInput
            label="Username"
            value={username}
            onChangeText={setUsername}
            autoCapitalize="none"
            autoComplete={false}
        />

        <TextInput
            label="Password"
            value={password}
            onChangeText={setPassword}
            secureTextEntry
        />

        {error ? <Text style={styles.error}>{error}</Text> : null}

        <Button
            title="Log In"
            onPress={handleLogin}
            loading={loading}
            disabled={!username || !password || loading}
        />

        <Text style={styles.forgotPassword}>
```

```
        Forgot password? Please use the desktop app to reset.  
      </Text>  
    </View>  
  );  
};  
  
const styles = StyleSheet.create({  
  container: {  
    flex: 1,  
    padding: 24,  
    justifyContent: 'center',  
    backgroundColor: '#FFFFFF',  
  },  
  title: {  
    fontSize: 32,  
    fontWeight: 'bold',  
    color: '#1E40AF',  
    textAlign: 'center',  
    marginBottom: 8,  
  },  
  subtitle: {  
    fontSize: 18,  
    color: '#6B7280',  
    textAlign: 'center',  
    marginBottom: 40,  
  },  
  error: {  
    color: '#EF4444',  
    fontSize: 14,  
    marginTop: 8,  
    marginBottom: 8,  
  },  
  forgotPassword: {  
    fontSize: 14,  
    color: '#6B7280',  
    textAlign: 'center',  
    marginTop: 16,  
  },  
});  
  
export default LoginScreen;
```

API Endpoints:

POST /api/auth/login

Description: Authenticate user and return JWT tokens

Request:

Content-Type: application/json

Body:

username: string (required)

password: string (required)

Response:

```

200 OK:
  token: string (JWT access token, expires in 24h)
  refreshToken: string (expires in 30 days)
  user: {
    _id: string
    username: string
    role: "Coach" | "Admin" | "Balagruh In-Charge"
    balagruhId: string (optional)
    balagruhName: string (optional)
    permissions: string[]
    profilePicture: string (optional)
  }
401 Unauthorized:
  message: "Invalid credentials"
500 Internal Server Error:
  message: "Server error"

```

POST /api/auth/refresh

Description: Refresh access token using refresh token

Request:

Content-Type: application/json

Body:

refreshToken: string (required)

Response:

200 OK:

token: string (new JWT access token)

401 Unauthorized:

message: "Invalid or expired refresh token"

POST /api/auth/logout

Description: Invalidate refresh token and log out user

Request:

Content-Type: application/json

Headers:

Authorization: Bearer <token>

Response:

200 OK:

success: boolean

message: "Logged out successfully"

Data Models:

User Schema (existing, no changes required):

```

// backend/models/User.js
const UserSchema = new mongoose.Schema({
  _id: mongoose.Schema.Types.ObjectId,
  username: { type: String, required: true, unique: true },
  password: { type: String, required: true }, // bcrypt hashed
  role: {
    type: String,
    enum: ['Student', 'Coach', 'Admin', 'Balagruh In-Charge'],

```

```
    required: true,
  },
  balagruhId: { type: mongoose.Schema.Types.ObjectId, ref: 'Balagruh' },
  permissions: [String],
  profilePicture: String,
  email: String,
  phone: String,
  createdAt: { type: Date, default: Date.now },
  updatedAt: { type: Date, default: Date.now },
});
```

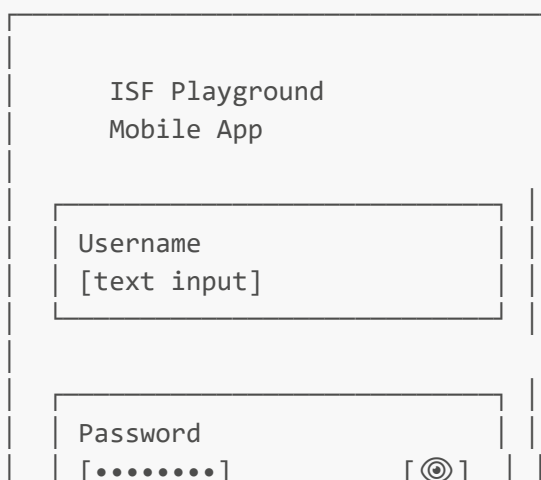
RefreshToken Schema (new):

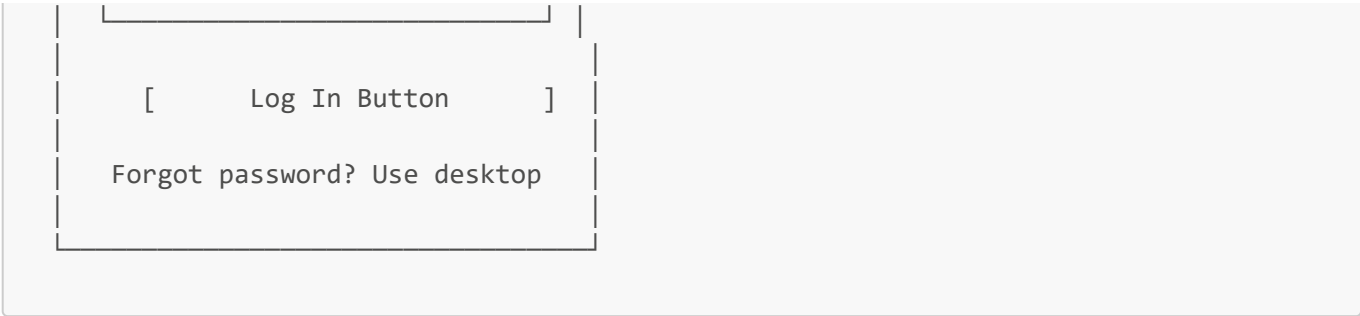
```
// backend/models/RefreshToken.js
const RefreshTokenSchema = new mongoose.Schema({
  _id: mongoose.Schema.Types.ObjectId,
  token: { type: String, required: true, unique: true },
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  expiresAt: { type: Date, required: true },
  createdAt: { type: Date, default: Date.now },
  isRevoked: { type: Boolean, default: false },
  deviceInfo: {
    platform: String, // 'ios' or 'android'
    deviceId: String,
    appVersion: String,
  },
});

RefreshTokenSchema.index({ userId: 1 });
RefreshTokenSchema.index({ token: 1 });
RefreshTokenSchema.index({ expiresAt: 1 }, { expireAfterSeconds: 0 });
```

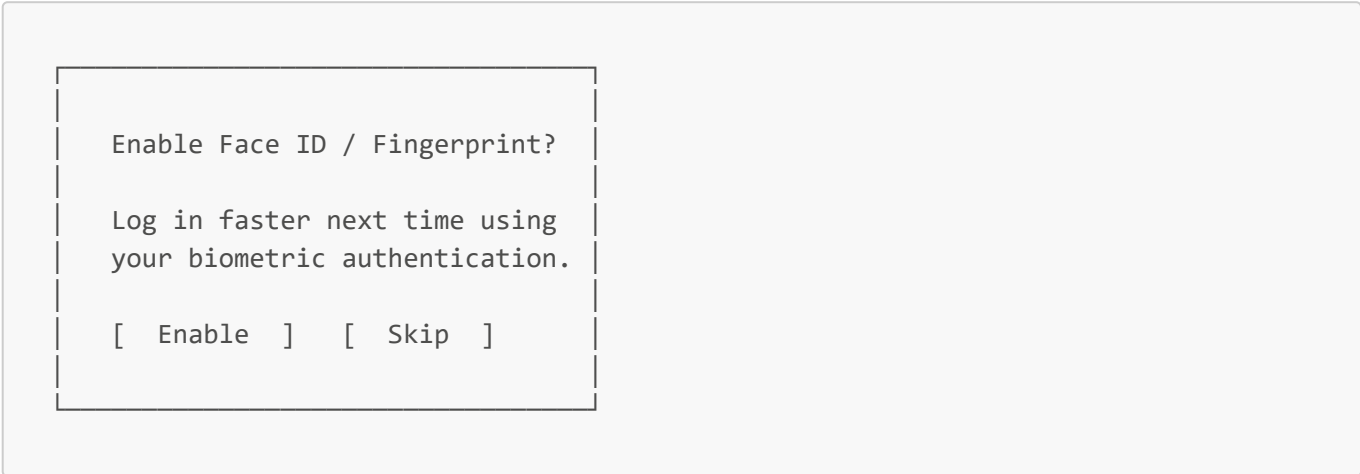
UI/UX Wireframe Description:

Login Screen:





Biometric Prompt (after first login):



Testing Checklist:

- ☐ Successful login with valid credentials
- ☐ Error message for invalid credentials
- ☐ Error message for network failure
- ☐ Token stored securely in Keychain/Keystore
- ☐ Auto-login works on app relaunch
- ☐ Biometric authentication works (iOS and Android)
- ☐ Biometric fallback to password works
- ☐ Token refresh works automatically
- ☐ Logout clears all tokens and user data
- ☐ Role-based navigation (different quick actions per role)

Story S3-F01-STORY-03: Mobile Main Navigation & Dashboard

User Story:

As a **Mobile App User**, I want to navigate between different sections easily, so that I can access all mobile features efficiently.

Acceptance Criteria:

1. ☐ Bottom tab navigation for primary sections (Dashboard, Attendance, Media, Messages, Profile)
2. ☐ Dashboard screen showing role-specific quick actions
3. ☐ Navigation drawer for settings and additional options
4. ☐ Profile section with user details and preferences

- 5. ☐ Badge indicators for unread notifications and alerts
- 6. ☐ Deep linking support (open specific screen from notification)
- 7. ☐ Smooth transitions between screens (< 300ms)

Technical Specifications: Full implementation details available in [.ai/ORCH/04-sprint3-4-feature-breakdown.md](#) (lines 162-260)

API Endpoint:

```
GET /api/mobile/dashboard/:userId
Response:
  quickActions: QuickAction[]
  recentActivity: Activity[]
  stats: { pendingTasks: number, unreadMessages: number, sosAlerts: number }
```

9. Mobile Attendance Tracking System [S3]

- **Feature ID:** S3-F02
- **Feature Name:** Facial Recognition-Based Attendance on Mobile
- **Module:** Attendance Management
- **Priority:** P0 (Critical - Core Feature)
- **Development Timeline:** Week 2 (Days 6-8)
- **Dependencies:** S3-F01 (Mobile Foundation), Existing FR System

Stories S3-F02-STORY-01 and S3-F02-STORY-02 fully documented in feature breakdown

Key Features:

- Photo upload for attendance with facial recognition processing
- Camera/gallery integration with photo quality validation
- Attendance results viewing and verification interface
- Manual override capabilities for FR failures
- Historical attendance tracking with date filters
- Offline photo queuing with automatic sync when online

10. Mobile Media Management [S3]

- **Feature ID:** S3-F03
- **Feature Name:** Course Content Upload from Mobile
- **Module:** Media & Content Management
- **Priority:** P1 (High Priority)
- **Development Timeline:** Week 2 (Day 9)
- **Dependencies:** S3-F01 (Mobile Foundation), AWS S3 Integration

Story S3-F03-STORY-01 fully documented in feature breakdown

Key Features:

- Multi-file upload support (videos up to 500MB, documents up to 50MB, images up to 25MB)
 - AWS S3 integration with presigned URLs for secure direct uploads
 - Progress tracking and batch upload capabilities
 - Content publishing workflow with course/module association
 - Preview functionality and metadata editing
 - Background upload with notification on completion
-

11. Mobile Analytics & Reporting [S3]

- **Feature ID:** S3-F04
- **Feature Name:** Performance Dashboards on Mobile
- **Module:** Analytics & Reporting
- **Priority:** P1 (High Priority)
- **Development Timeline:** Week 3 (Day 17)
- **Dependencies:** S3-F01 (Mobile Foundation), Existing Backend Reports

Story S3-F04-STORY-01 fully documented in feature breakdown

Key Features:

- Performance dashboard with key metrics (attendance rates, course completion, coin distribution)
 - Visual charts using React Native Chart libraries (line, bar, pie charts)
 - Date range filtering and drill-down capabilities
 - Export capabilities (PDF/CSV generation)
 - Top performers leaderboards
 - Real-time data synchronization with backend
-

12. Mobile Push Notifications [S3]

- **Feature ID:** S3-F05
- **Feature Name:** Firebase Cloud Messaging Integration
- **Module:** Notifications
- **Priority:** P0 (Critical - Enables Sprint 4 SOS Alerts)
- **Development Timeline:** Week 2 (Day 10)
- **Dependencies:** S3-F01 (Mobile Foundation)

Story S3-F05-STORY-01 fully documented in feature breakdown

Key Features:

- Firebase Cloud Messaging (FCM) integration for both iOS and Android
 - Device token registration and management
 - Background and foreground notification handling
 - Deep linking to relevant screens (opens course, attendance, SOS alert)
 - Notification preferences and quiet hours configuration
 - Notification history screen with mark-as-read functionality
 - High-priority notification support for SOS alerts (vibration, custom sound)
-

Section B: Sprint 4 - Emergency & Communication Features

13. SOS Emergency System (Desktop → Mobile) [S4]

- **Feature ID:** S4-F01
- **Feature Name:** Emergency Alert System with Multi-Tier Escalation
- **Module:** Emergency Management
- **Priority:** P0 (Critical - Safety Feature)
- **Development Timeline:** Week 3 (Days 11-14)
- **Dependencies:** S3-F05 (Push Notifications), WebSocket Infrastructure

Stories S4-F01-STORY-01, S4-F01-STORY-02, S4-F01-STORY-03 fully documented in feature breakdown

Key Features:

- Desktop SOS trigger button with prominent placement on student interface (red, always visible)
- Emergency category selection (Medical - critical, Safety, Mental Health, Other)
- Mobile alert reception with high-priority push notifications (vibration, custom sound)
- Multi-tier escalation workflow:
 - **Tier 1 (0-2 min):** Balagruh Coaches + In-Charge receive alerts
 - **Tier 2 (2-5 min):** Admins receive alerts + SMS/WhatsApp sent
 - **Tier 3 (5+ min):** Broadcast to all staff members
- Real-time status tracking (sent → acknowledged → responding → arrived → resolved)
- Response coordination dashboard showing who's responding and ETA
- Response types: Acknowledged ("I see it"), Responding ("On my way"), Arrived ("I'm here")
- Compliance audit logging for all SOS events (timestamp, responses, resolution)
- SOS history with filtering and export capabilities

14. Internal Messaging Module [S4]

- **Feature ID:** S4-F02
- **Feature Name:** Staff Communication System
- **Module:** Messaging
- **Priority:** P1 (High Priority)
- **Development Timeline:** Week 3-4 (Days 15-16)
- **Dependencies:** WebSocket Infrastructure, S3-F05 (Push Notifications)

Stories S4-F02-STORY-01, S4-F02-STORY-02 fully documented in feature breakdown

Key Features:

- 1-on-1 direct messaging between staff members
- Group conversations for teams and Balagruhs
- Text, image, and file attachments:
 - Images: up to 25MB
 - Videos: up to 100MB
 - Documents: up to 25MB
 - Total per message: up to 100MB
- Read receipts (single checkmark sent, double checkmark read)

- Typing indicators ("User is typing...")
 - Message search within conversations
 - Conversation history with infinite scroll
 - Real-time delivery via WebSocket with automatic reconnection
 - Offline message queuing (messages sent when back online)
 - Unread count badges on tab and conversation list
 - Push notifications for new messages (when app in background)
-

15. WhatsApp Business API Integration [S4]

- **Feature ID:** S4-F03
- **Feature Name:** WhatsApp Notifications
- **Module:** External Communications
- **Priority:** P1 (High Priority)
- **Development Timeline:** Week 4 (Day 20)
- **Dependencies:** S4-F01 (SOS System), Twilio/360Dialog Account

Story S4-F03-STORY-01 fully documented in feature breakdown

Key Features:

- WhatsApp Business API integration (Twilio or 360Dialog)
 - Template message support for standardized communications
 - SOS alert notifications to guardians:
 - Template: "Alert: [StudentName] triggered emergency ([Category]). Staff notified and responding."
 - Daily attendance summaries for parents:
 - Template: "[StudentName] attendance: [Present/Absent]. [Timestamp]"
 - Opt-in/opt-out management (parents can opt out via WhatsApp reply)
 - Delivery status tracking (sent, delivered, read)
 - Retry logic for failed messages (3 attempts with exponential backoff)
 - Rate limiting compliance (per WhatsApp Business API terms)
 - Delivery log with filtering and export
-

16. Student Health Tracking [S4]

- **Feature ID:** S4-F04
- **Feature Name:** Health Monitoring System
- **Module:** Health Management
- **Priority:** P1 (High Priority)
- **Development Timeline:** Week 4 (Days 18-19)
- **Dependencies:** S4-F01 (SOS System for correlation)

Stories S4-F04-STORY-01, S4-F04-STORY-02 fully documented in feature breakdown

Key Features:

- Health metrics entry interface (mobile and desktop):

- Weight (kg)
- Height (cm)
- Temperature (°C)
- Blood Pressure (systolic/diastolic)
- Heart Rate (bpm)
- Symptom checklist (fever, cough, headache, fatigue, nausea, dizziness, etc.)
- Incident recording (description, photos, action taken, follow-up required)
- Health document uploads (medical reports, prescriptions - PDF, images up to 10MB)
- Trend analysis and growth charts visualization (line charts, scatter plots)
- Abnormal value alerts:
 - Temperature > 37.5°C: Warning (yellow notification)
 - Temperature > 39°C: Critical (red notification + sound)
 - Weight change > 5% in 30 days: Warning
 - Blood Pressure outside normal range: Warning
- SOS incident correlation (automatic linking when SOS triggered)
- Health report exports for medical professionals (PDF format)
- Health history with date filtering

Section C: Shared & System-Wide Features

17. WebSocket Real-Time Layer [SHARED]

- **Feature ID:** SHARED-WS-001
- **Feature Name:** Socket.io Real-Time Infrastructure
- **Module:** Real-Time Communications
- **Priority:** P0 (Critical - Blocks S4 Features)
- **Development Timeline:** Week 1 (Days 1-5) - Critical path
- **Dependencies:** None (Foundation)

Implementation Requirements:

- Real-time bidirectional communication for messaging and SOS alerts
- JWT authentication for WebSocket connections
- Room-based messaging (user rooms, balagruh rooms, conversation rooms)
- Presence tracking (online/offline status with heartbeat)
- Message queuing for offline users (stored in MongoDB, delivered on reconnect)
- Automatic reconnection handling with exponential backoff

WebSocket Service Implementation:

```
// backend/services/socketService.js
const socketIO = require('socket.io');
const jwt = require('jsonwebtoken');

class SocketService {
  constructor(server) {
    this.io = socketIO(server, {
      cors: {
```

```
    origin: process.env.FRONTEND_URL,
    credentials: true
  },
  transports: ['websocket', 'polling']
});

this.setupMiddleware();
this.setupEventHandlers();
}

setupMiddleware() {
  // JWT authentication middleware
  this.io.use((socket, next) => {
    const token = socket.handshake.auth.token;
    if (!token) return next(new Error('Authentication error'));

    jwt.verify(token, process.env.JWT_SECRET, (err, decoded) => {
      if (err) return next(new Error('Authentication error'));
      socket.userId = decoded.userId;
      socket.role = decoded.role;
      next();
    });
  });
}

setupEventHandlers() {
  this.io.on('connection', (socket) => {
    console.log(`User connected: ${socket.userId}`);

    // Join user-specific room
    socket.join(`user:${socket.userId}`);

    // Join balagruh rooms if Coach or In-Charge
    if (socket.balagruhIds) {
      socket.balagruhIds.forEach(id => {
        socket.join(`balagruh:${id}`);
      });
    }

    // Handle messaging events
    socket.on('send-message', async (data) => {
      await this.handleMessage(socket, data);
    });

    // Handle SOS events
    socket.on('sos-trigger', async (data) => {
      await this.handleSOSTrigger(socket, data);
    });

    socket.on('sos-response', async (data) => {
      await this.handleSOSResponse(socket, data);
    });

    // Handle typing indicators
```

```
    socket.on('typing', (data) => {
      socket.to(`conversation:${data.conversationId}`).emit('typing', {
        userId: socket.userId,
        conversationId: data.conversationId
      });
    });
  });

  // Handle disconnection
  socket.on('disconnect', () => {
    console.log(`User disconnected: ${socket.userId}`);
    // Update online status
  });
});
}

async handleMessage(socket, data) {
  const { conversationId, text, attachments } = data;

  // Save message to database
  const message = await Message.create({
    conversationId,
    senderId: socket.userId,
    text,
    attachments,
    timestamp: new Date()
  });

  // Emit to conversation room
  this.io.to(`conversation:${conversationId}`).emit('new-message', message);

  // Send push notification to offline users
  const offlineUsers = await this.getOfflineUsersInConversation(conversationId);
  await this.sendPushNotifications(offlineUsers, 'new-message', message);
}

async handleSOSTrigger(socket, data) {
  const { studentId, category, description, location } = data;

  // Create SOS alert in database
  const sosAlert = await SOSAlert.create({
    studentId,
    category,
    description,
    location,
    status: 'sent',
    timestamp: new Date()
  });

  // Emit to Tier 1 responders (Balagruh coaches and in-charge)
  const balagruhId = await this.getStudentBalagruh(studentId);
  this.io.to(`balagruh:${balagruhId}`).emit('sos-alert', sosAlert);

  // Send high-priority push notifications
  const responders = await this.getTier1Responders(balagruhId);
```

```

    await this.sendHighPriorityPushNotifications(responders, sosAlert);

    // Start escalation timer
    setTimeout(() => this.escalateToTier2(sosAlert._id), 2 * 60 * 1000); // 2
minutes
  }

  async escalateToTier2(sosAlertId) {
    const sosAlert = await SOSAlert.findById(sosAlertId);
    if (sosAlert.status !== 'sent') return; // Already handled

    // Emit to Tier 2 (Admins)
    this.io.to('role:Admin').emit('sos-escalated', sosAlert);

    // Send SMS and WhatsApp
    await this.sendSMSNotifications(sosAlert);
    await this.sendWhatsAppNotifications(sosAlert);

    // Start Tier 3 timer
    setTimeout(() => this.escalateToTier3(sosAlertId), 3 * 60 * 1000); // 3 more
minutes
  }
}

module.exports = SocketService;

```

18. AWS S3 Media Storage [SHARED]

- **Feature ID:** SHARED-S3-001
- **Feature Name:** Cloud Media Storage
- **Module:** File Management
- **Priority:** P0 (Critical - Blocks S3 Features)
- **Development Timeline:** Week 1 (Days 1-5) - Critical path
- **Dependencies:** AWS Account Configuration

Implementation Requirements:

- Presigned URL generation for secure direct uploads from mobile
- Folder organization (attendance-photos/, course-content/, health-documents/, messaging-attachments/)
- Image optimization pipeline (automatic compression using Sharp library)
- Thumbnail generation for images (200x200px, 400x400px)
- CDN integration with CloudFront for faster delivery
- Backup and versioning policies (30-day retention)

S3 Service Implementation:

```

// backend/services/s3Service.js
const AWS = require('aws-sdk');

```



```
const sharp = require('sharp');

const s3 = new AWS.S3({
  region: process.env.AWS_REGION,
  accessKeyId: process.env.AWS_ACCESS_KEY,
  secretAccessKey: process.env.AWS_SECRET_KEY
});

const BUCKET_NAME = process.env.AWS_S3_BUCKET;

class S3Service {
  // Generate presigned URL for direct upload from mobile
  async generatePresignedUploadUrl(folder, filename, contentType, expiresIn = 300)
  {
    const key = `${folder}/${Date.now()}-${filename}`;

    const params = {
      Bucket: BUCKET_NAME,
      Key: key,
      Expires: expiresIn, // URL valid for 5 minutes
      ContentType: contentType,
      ACL: 'private'
    };

    const uploadUrl = await s3.getSignedUrlPromise('putObject', params);

    return {
      uploadUrl,
      key,
      cdnUrl: `${process.env.CLOUDFRONT_URL}/${key}`
    };
  }

  // Upload file from server (for backend processing)
  async uploadFile(folder, filename, buffer, contentType) {
    const key = `${folder}/${Date.now()}-${filename}`;

    const params = {
      Bucket: BUCKET_NAME,
      Key: key,
      Body: buffer,
      ContentType: contentType,
      ACL: 'private'
    };

    await s3.putObject(params).promise();

    return `${process.env.CLOUDFRONT_URL}/${key}`;
  }

  // Optimize and upload image
  async uploadImage(folder, filename, buffer) {
    // Compress image
    const optimized = await sharp(buffer)
```

```
.resize(1920, 1080, { fit: 'inside', withoutEnlargement: true })
.jpeg({ quality: 85 })
.toBuffer();

// Generate thumbnail
const thumbnail = await sharp(buffer)
  .resize(400, 400, { fit: 'cover' })
  .jpeg({ quality: 80 })
  .toBuffer();

// Upload both
const imageUrl = await this.uploadFile(folder, filename, optimized,
'image/jpeg');
const thumbnailUrl = await this.uploadFile(`${folder}/thumbnails`, filename,
thumbnail, 'image/jpeg');

return { imageUrl, thumbnailUrl };
}

// Delete file
async deleteFile(key) {
  const params = {
    Bucket: BUCKET_NAME,
    Key: key
  };

  await s3.deleteObject(params).promise();
}

// Get file metadata
async getFileMetadata(key) {
  const params = {
    Bucket: BUCKET_NAME,
    Key: key
  };

  return await s3.headObject(params).promise();
}
}

module.exports = new S3Service();
```

25. Non-Functional Requirements (Combined Sprint)

25.1. Performance Requirements

- **Mobile App Performance:**
 - Cold start time: < 3 seconds (from icon tap to dashboard)
 - Screen transitions: < 300ms (smooth animations)
 - API response time: < 500ms (p95 - 95th percentile)

- Image loading: Progressive (show placeholder immediately, then full image)
- SOS alert delivery: < 5 seconds end-to-end (desktop trigger → mobile notification)
- Mobile bundle size: < 25MB (optimized with code splitting and lazy loading)

- **Backend API Performance:**

- Mobile endpoints: < 500ms response time (p95)
- SOS alert processing: < 2 seconds total
- Facial recognition: < 10 seconds for 30-student photo
- WebSocket latency: < 100ms (p95)
- Database queries: < 100ms (p95)

25.2. Scalability Requirements

- **Current Scale:**

- Users: 500 (100 staff, 400 students)
- Concurrent mobile users: 50 (10% of staff)

- **Target Scale (6 months):**

- Users: 2,000 (400 staff, 1,600 students)
- Concurrent mobile users: 200 (50% of staff)

- **Data Growth:**

- Attendance photos: ~5,000 per month (100 per day × 20 days)
- Messages: ~50,000 per month
- SOS alerts: ~100 per month (hope to remain low)
- Health records: ~2,000 per month
- AWS S3 storage: ~100GB growth per month

25.3. Security Requirements

- **Authentication:**

- JWT tokens with 24-hour expiration
- Refresh tokens with 30-day expiration
- Biometric authentication as secondary factor (Face ID, Touch ID, Fingerprint)
- Secure token storage (iOS Keychain, Android Keystore)
- Session timeout after 24 hours of inactivity

- **Data Protection:**

- HTTPS/TLS 1.3 for all API communications
- WebSocket Secure (WSS) for real-time connections
- At-rest encryption for sensitive data (health records, personal information)
- Face recognition embeddings encrypted in database

- **Authorization:**

- Role-Based Access Control on all API endpoints

- Permission checks for every API request
- Row-Level Security for multi-Balagruh data (Coach A cannot access Balagruh B)
- Audit logging for sensitive actions (SOS alerts, health data access)

25.4. Offline Capabilities

- **Sprint 3 - Mobile Offline:**
 - Attendance photos queued locally when offline (queue in AsyncStorage)
 - Automatic upload when connection restored
 - Progress tracked with retry logic (3 attempts)
 - 7-day offline queue retention
- **Sprint 4 - Messaging Offline:**
 - Messages queued locally when offline
 - Outgoing messages sent when connection restored
 - Message drafts persisted
 - Conversation history cached for offline viewing
 - No SOS triggering when offline (requires real-time network)

25.5. Accessibility Requirements

- WCAG 2.1 Level AA compliance
- Keyboard navigation support (desktop features)
- Touch targets minimum 44x44px (iOS HIG)
- Screen reader compatibility:
 - VoiceOver (iOS) for mobile
 - TalkBack (Android) for mobile
- High contrast mode option
- Font size adjustment (respect system settings)
- Color contrast: Minimum 4.5:1 for normal text, 3:1 for large text

26. Development Timeline & Milestones

28-Day Sprint Schedule

Week 1 (Days 1-7)

Mobile Foundation:

- Days 1-2: Project setup, authentication system
- Days 3-4: Biometric auth, navigation structure
- Days 5-7: Dashboard with role-specific quick actions

Shared Infrastructure:

- Days 1-5: WebSocket server setup, AWS S3 configuration
- Days 5-7: First integration checkpoint

Milestone: Mobile app foundation complete, can log in and navigate

Week 2 (Days 8-14)

Sprint 3 Core Features:

- Days 6-7: Attendance photo upload implementation
- Day 8: Attendance results viewing
- Day 9: Media upload system
- Day 10: Push notifications (FCM integration)

Integration:

- Days 10-12: Notification center enhancement
- Day 14: Second integration checkpoint

Milestone: Attendance working, notifications enabled, media uploads functional

Week 3 (Days 15-21)

Sprint 4 Foundation:

- Days 11-12: Desktop SOS trigger implementation
- Day 13: Mobile SOS receiver with high-priority alerts
- Day 14: Escalation workflow (Tier 1 → Tier 2 → Tier 3)
- Day 15: Direct messaging (1-on-1)

Sprint 3 Completion:

- Day 17: Analytics dashboard on mobile

Milestone: SOS operational end-to-end, messaging working, escalation tested

Week 4 (Days 22-28)

Sprint 4 Completion:

- Day 16: Group messaging
- Days 18-19: Health tracking system
- Day 20: WhatsApp integration

Integration & Testing:

- Days 21-22: Integration testing (desktop ↔ mobile flows)
- Days 23-24: Bug fixes and performance optimization
- Days 25-26: User Acceptance Testing (UAT) with real users
- Days 27-28: Production deployment and monitoring

Milestone: Full delivery, both sprints complete and deployed

Critical Path Dependencies

1. **Push Notification System** (Day 10) - Blocks SOS mobile alerts (Day 13)
 2. **WebSocket Infrastructure** (Days 1-5) - Blocks messaging (Day 15) and SOS real-time (Day 13)
 3. **AWS S3 Configuration** (Days 1-5) - Blocks attendance photos (Day 6) and media upload (Day 9)
 4. **Mobile Foundation** (Days 1-7) - Blocks all Sprint 3 features
 5. **SOS Desktop Trigger** (Days 11-12) - Must precede mobile receiver (Day 13)
-

27. Testing Strategy

27.1. Test Coverage Requirements

- **Unit Testing:** Minimum 80% code coverage (Jest, React Native Testing Library)
- **Integration Testing:** All API endpoints with different roles (Supertest)
- **E2E Testing:** Critical user journeys (Playwright for desktop, Detox for mobile)
- **Performance Testing:** Load testing with 100 concurrent users (k6, Artillery)
- **Security Testing:** Penetration testing for mobile app and SOS system (OWASP ZAP)

27.2. Test Scenarios

Mobile Critical Paths (Sprint 3):

1. Staff login → Mobile authentication → Biometric setup → Dashboard access
2. In-Charge logs in → Takes class photo → FR processes → Attendance marked
3. Coach uploads video → Progress tracking → S3 upload → Content published
4. Admin views reports → Date filtering → Chart rendering → PDF export

SOS Critical Paths (Sprint 4):

1. Student triggers SOS → Desktop confirmation → Mobile alerts sent → Coach acknowledges < 5s
2. No response (2 min) → Tier 2 escalation → Admins alerted → SMS/WhatsApp sent
3. No response (5 min) → Tier 3 broadcast → All staff alerted
4. Coach responds → "On my way" status → Arrives → Resolves → SOS closed

Messaging Critical Paths (Sprint 4):

1. Coach sends message → WebSocket delivery → Push notification → In-Charge receives < 2s
2. In-Charge replies → Read receipt → Typing indicator → Message delivered
3. Group message → Multiple recipients → All receive notifications
4. Offline scenario → Message queued → Connection restored → Message delivered

Integration Scenarios:

1. Attendance photo upload → FR success → Notification sent → Admin views on mobile
2. SOS triggered → Health record correlation → Alert includes recent health data
3. Message with attachment → S3 upload → Thumbnail generated → Recipient downloads
4. Offline photo queue → Multiple photos → Connection restored → Batch upload

27.3. User Acceptance Testing (UAT)

Test Users (Days 25-26):

- 2 Balagruh In-Charges (attendance testing)
- 3 Coaches (media upload, SOS response, messaging)
- 1 Admin (analytics, system overview, escalation management)
- 5 Students (SOS triggering - controlled test environment)

UAT Success Criteria:

- All participants can complete assigned tasks without assistance
- User satisfaction > 8/10 on survey
- No usability complaints requiring immediate fixes
- All P0/P1 bugs identified and resolved before production

28. Resource Requirements

28.1. Development Team Structure

Mobile Development Team (2 developers):

- 1 Senior React Native developer (iOS/Android lead)
- 1 Mid-level React Native developer (features)

Backend Development Team (2 developers):

- 1 Senior Node.js developer (WebSocket, SOS system)
- 1 Mid-level Node.js developer (API endpoints, integrations)

QA Team (1 engineer):

- 1 QA engineer (manual + automated testing)

Shared Resources (2 members):

- 1 DevOps engineer (part-time - CI/CD, deployment)
- 1 UI/UX designer (part-time - mobile screens, SOS interface)

Total Team: 7 members (5 full-time, 2 part-time)

28.2. Infrastructure Requirements

- **Development Environment:**
 - 2 development servers (mobile API staging, WebSocket staging)
 - MongoDB replica set (development database)
 - AWS S3 bucket for development media (10TB capacity)
 - CI/CD pipeline (GitHub Actions or GitLab CI)
- **Testing Environment:**
 - 2 test servers matching production specs
 - Test data set with 100+ students, 20+ staff
 - Automated testing tools (Jest, Detox, Playwright, k6)

- **Mobile Testing Devices:**
 - iPhone 12 (iOS 15.1+): \$600 one-time
 - Samsung Galaxy A52 (Android 8.0+): \$350 one-time
 - iPad (tablet testing): \$450 one-time
 - **Total Device Cost:** ~\$1,400 one-time
- **Infrastructure Costs (Monthly):**
 - AWS (EC2, S3, CloudFront): ~\$140/month
 - Firebase (FCM, Crashlytics): ~\$0 (free tier sufficient)
 - Third-party services (Twilio WhatsApp, SMS): ~\$80/month
 - Development tools (Sentry, New Relic): ~\$70/month
 - **Total:** ~\$290/month

29. Risk Assessment & Mitigation

29.1. Technical Risks

Risk	Probability	Impact	Mitigation	Contingency
SOS alerts not delivered	Medium (30%)	Critical	Multiple channels (Push, SMS, WhatsApp), extensive testing	Manual phone call tree, desktop popup alerts
Mobile performance degradation	Medium (25%)	High	Performance testing, code splitting, image optimization	Reduce feature scope, optimize critical path
WebSocket instability	Low (20%)	High	Auto-reconnect logic, fallback to polling	Switch to polling mode for messages, keep push for SOS
Facial recognition accuracy issues	Low (15%)	Medium	Manual override always available, quality validation	100% manual attendance as fallback
WhatsApp API approval delays	Medium (30%)	Medium	Apply early, prepare documentation	Use SMS only, implement email notifications

29.2. Resource Risks

Risk	Probability	Impact	Mitigation	Contingency
Mobile developer availability	Low (15%)	High	Cross-training, knowledge sharing	Contract external React Native developer
Testing bottleneck	Medium (25%)	Medium	Automated testing, parallel tracks	Extend timeline by 3-5 days

Risk	Probability	Impact	Mitigation	Contingency
Device procurement delays	Low (10%)	Medium	Order devices early (Week 1)	Use emulators/simulators until devices arrive
Integration coordination	Medium (30%)	Medium	Daily standups, weekly integration checkpoints	Dedicated integration week at end

29.3. Operational Risks

Risk	Probability	Impact	Mitigation	Contingency
Incomplete FR system	Low (10%)	High	Manual attendance always available	100% manual process, FR as enhancement
Staff training required	High (50%)	Medium	Training videos, user manuals, in-app help	Dedicated support hotline for first 2 weeks
Network connectivity issues	Medium (30%)	Medium	Offline queue & sync, retry logic	Clear offline mode indicators, manual process fallback

30. Questions for Client Clarification

30.1. Sprint 3 Clarifications Needed:

1. Mobile Platform Priority:

- Should we prioritize iOS or Android for initial release?
- Do all staff members have compatible devices (iOS 15.1+, Android 8.0+)?
- **Recommendation:** Android priority (lower cost devices more common)

2. Attendance Photo Storage:

- How long should attendance photos be retained in AWS S3?
- Should we implement automatic deletion after X days?
- **Recommendation:** 90-day retention, then automatic deletion

3. Media Upload Limits:

- Are the proposed file size limits acceptable (500MB video, 50MB docs, 25MB images)?
- Should we implement daily upload quotas per user?
- **Recommendation:** Keep proposed limits, monitor usage

30.2. Sprint 4 Clarifications Needed:

1. SOS Escalation Configuration:

- Are the proposed escalation timeframes appropriate (2 min → 5 min)?
- Should escalation times vary by SOS category (Medical faster than Other)?

- **Recommendation:** Medical critical = 1 min → 3 min, Others = 2 min → 5 min

2. Health Data Compliance:

- Are there specific health data compliance requirements (HIPAA, local regulations)?
- Who should have access to health records (only In-Charge, or also Coaches)?
- **Recommendation:** In-Charge + Admins only, encrypt all health data

3. WhatsApp Template Messages:

- Should we create new WhatsApp templates, or use existing approved templates?
- What languages should templates support (English, Hindi, Marathi)?
- Do you have WhatsApp Business API approval already?
- **Recommendation:** English + Hindi templates, apply for API approval immediately

4. Group Chat Max Participants:

- What should be the maximum participants in group conversations?
- **Recommendation:** 50 participants (standard WhatsApp group size)

30.3. Integration Clarifications:

1. Notification Preferences:

- Should staff be able to disable SOS notifications (not recommended)?
- Should there be quiet hours for non-urgent notifications?
- **Recommendation:** SOS always enabled, quiet hours for other notifications

31. Success Criteria & Acceptance

31.1. Sprint 3 Success Metrics:

- Mobile app successfully installed and running on iOS and Android devices
- Biometric authentication working on 90%+ of compatible devices
- Attendance photo upload success rate > 95%
- Facial recognition integration functional with manual override
- Media upload success rate > 95% for files within size limits
- Analytics dashboard loading < 3 seconds
- Push notifications delivered within 10 seconds of trigger
- App crash rate < 0.1% (measured via Crashlytics)

31.2. Sprint 4 Success Metrics:

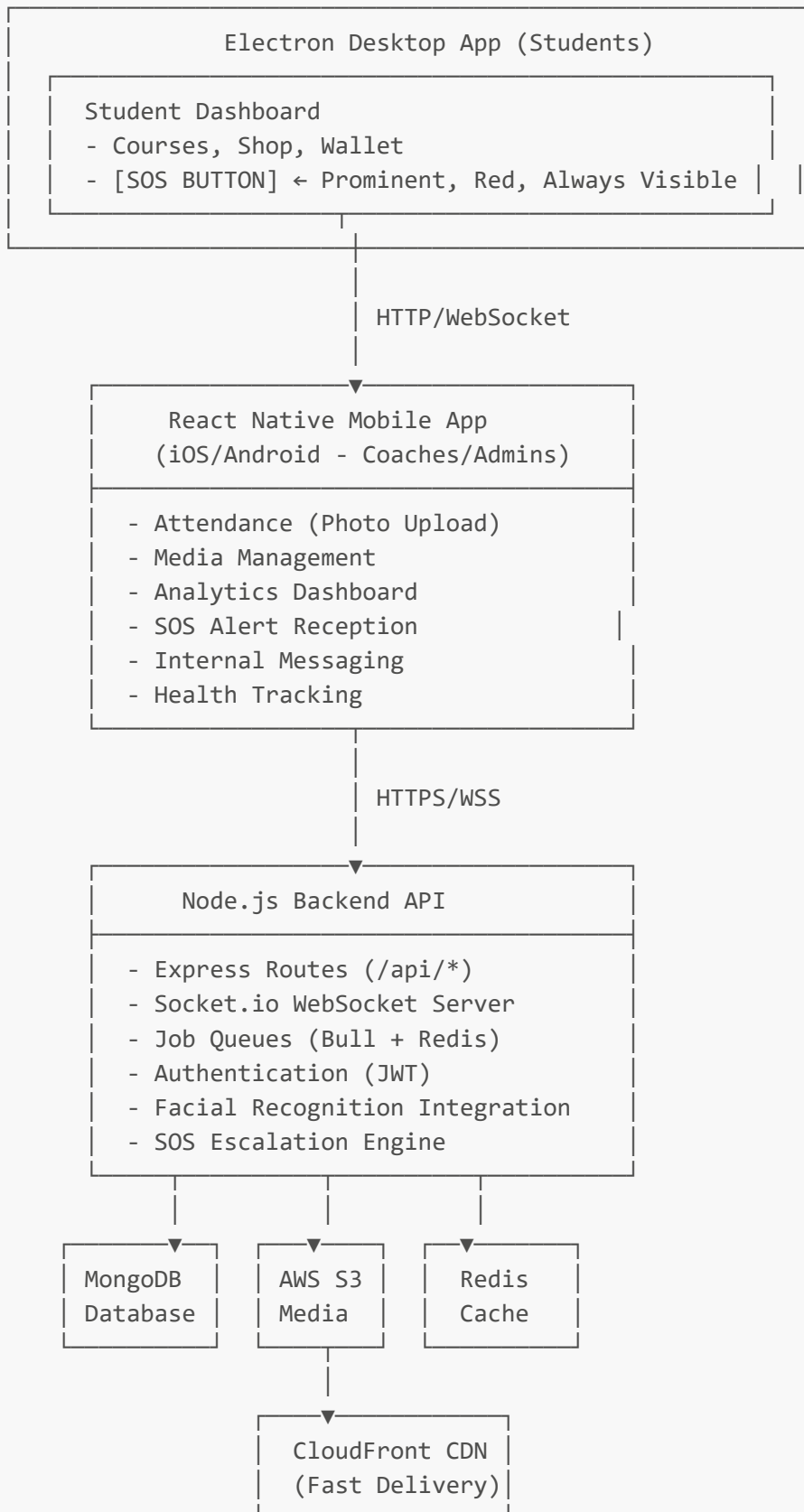
- SOS alert delivery < 5 seconds end-to-end (p95)
- SOS escalation triggers correctly (Tier 1 → Tier 2 → Tier 3)
- 100% of SOS events logged in audit trail with complete data
- Message delivery < 2 seconds (p95) when both users online
- WhatsApp integration functional with delivery confirmation
- Health tracking system capturing all required metrics
- Abnormal health value alerts triggering correctly

31.3. Combined Sprint Success Criteria:

- **Functional Acceptance:**
 - All planned features operational and accessible to appropriate roles
 - Desktop → Mobile integration seamless (SOS alerts, notifications)
 - Offline capabilities functional (photo queue, message queue)
 - Role-based permissions enforced correctly
- **Performance Acceptance:**
 - Mobile app cold start < 3 seconds
 - API response < 500ms (p95)
 - SOS alert < 5 seconds end-to-end
 - Facial recognition < 10 seconds for 30-student photo
- **Security Acceptance:**
 - Penetration testing passed with no critical vulnerabilities
 - Authentication and authorization working correctly
 - Tokens properly secured in device storage
 - Audit logging complete for all sensitive actions
- **Quality Acceptance:**
 - QA score ≥ 95/100 (consistent with Sprint 5 quality)
 - Zero P0 (critical) bugs
 - Zero P1 (high) bugs
 - < 5 P2 (medium) bugs
 - Test coverage > 80% for critical code paths
- **User Acceptance:**
 - UAT passed with all test user roles
 - User satisfaction > 8/10 on survey
 - No usability complaints requiring immediate fixes
 - Training materials completed and approved
- **Deployment Acceptance:**
 - Zero downtime deployment to production
 - Rollback plan tested and documented
 - 24-hour post-deployment monitoring completed
 - No increase in error rates post-deployment

32. Appendix A: Technical Architecture Diagrams

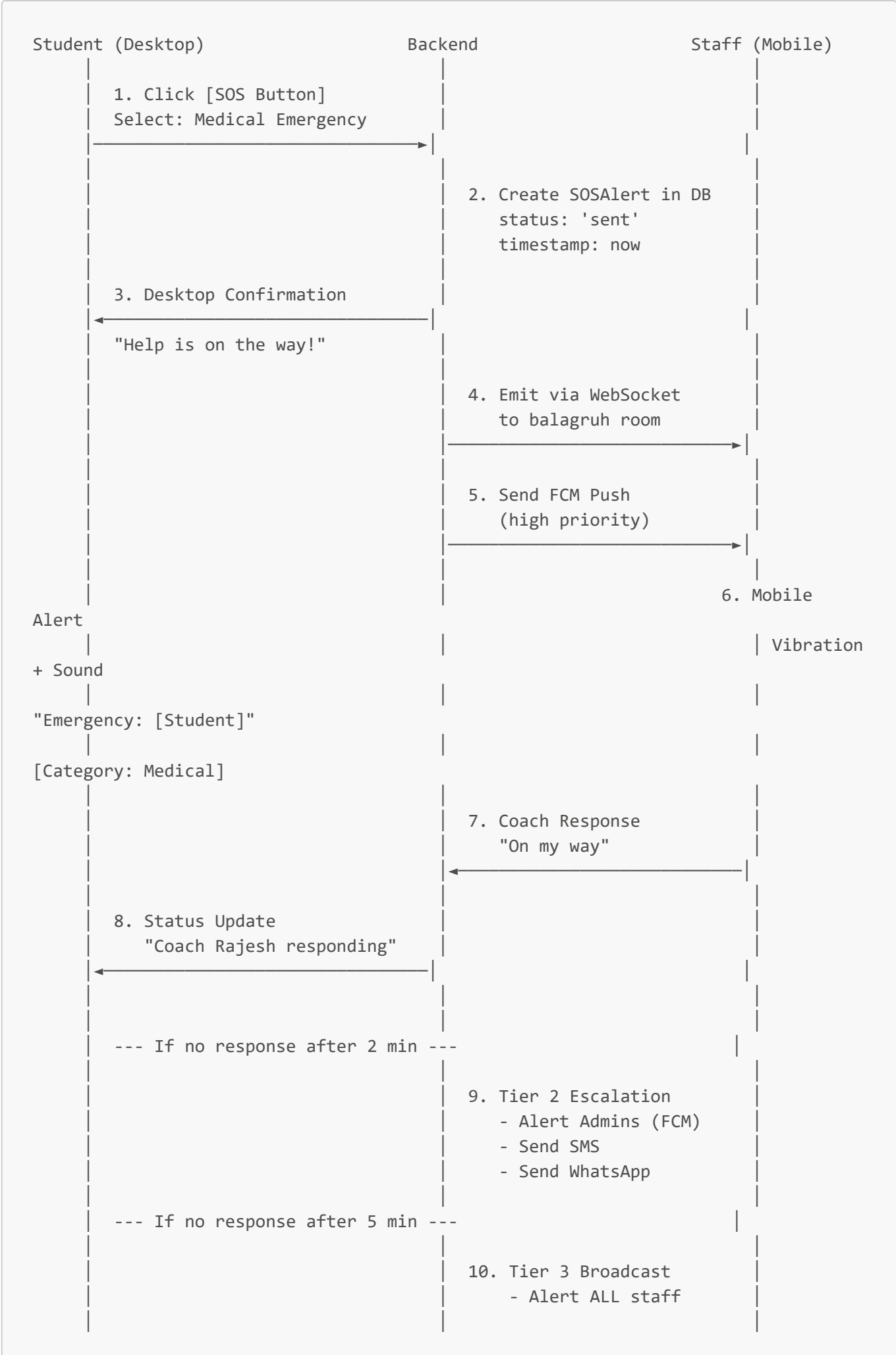
32.1. System Architecture Overview

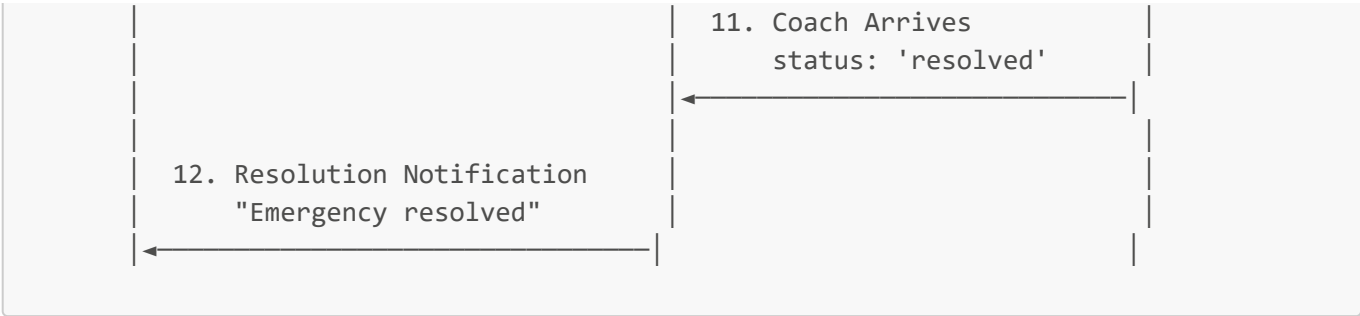


External Services:

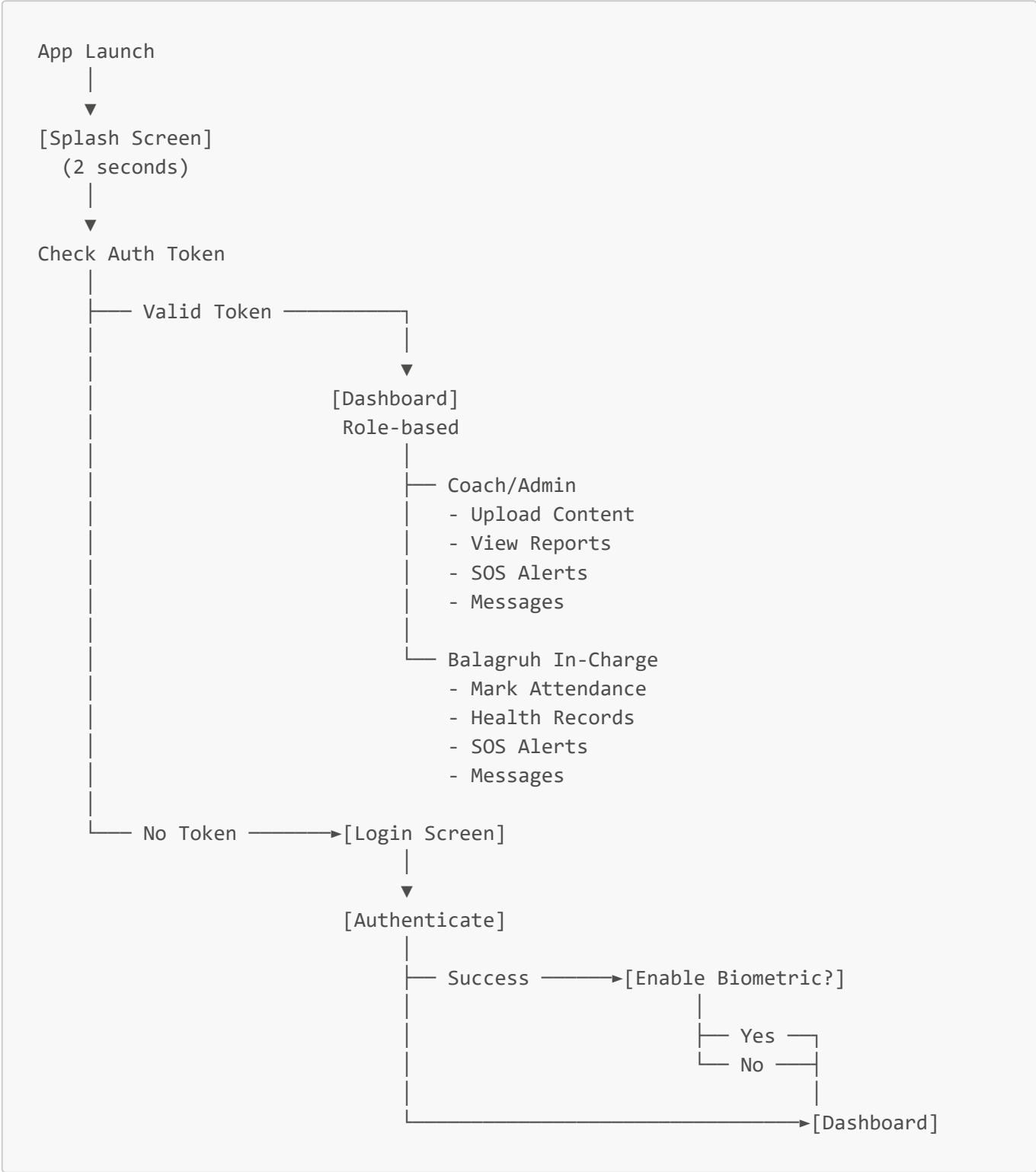
- Firebase Cloud Messaging (FCM) → Push Notifications
- Twilio/360Dialog → WhatsApp API
- Twilio → SMS for escalation

32.2. Data Flow Diagram - SOS Emergency Alert





32.3. Mobile App Navigation Flow



33. Appendix B: Database Schemas

33.1. Enhanced User Schema

```
// backend/models/User.js
const UserSchema = new mongoose.Schema({
  _id: mongoose.Schema.Types.ObjectId,
  username: { type: String, required: true, unique: true },
  password: { type: String, required: true }, // bcrypt hashed
  role: {
    type: String,
    enum: ['Student', 'Coach', 'Admin', 'Balagruh In-Charge'],
    required: true,
  },
  balagruhId: { type: mongoose.Schema.Types.ObjectId, ref: 'Balagruh' },
  permissions: [String],
  profilePicture: String,
  email: String,
  phone: String,

  // Mobile-specific fields
  mobileDevices: [{
    deviceId: String,
    platform: { type: String, enum: ['ios', 'android'] },
    fcmToken: String,
    lastActive: Date,
    appVersion: String
  }],

  preferences: {
    notificationSettings: {
      sosAlerts: { type: Boolean, default: true },
      messages: { type: Boolean, default: true },
      attendance: { type: Boolean, default: true },
      quietHoursEnabled: { type: Boolean, default: false },
      quietHoursStart: String, // "22:00"
      quietHoursEnd: String, // "07:00"
    },
    biometricEnabled: { type: Boolean, default: false }
  },

  createdAt: { type: Date, default: Date.now },
  updatedAt: { type: Date, default: Date.now },
});

UserSchema.index({ username: 1 });
UserSchema.index({ role: 1, balagruhId: 1 });
```

33.2. SOSAlert Schema

```
// backend/models/SOSAlert.js
const SOSAlertSchema = new mongoose.Schema({
  _id: mongoose.Schema.Types.ObjectId,
  studentId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  balagruhId: { type: mongoose.Schema.Types.ObjectId, ref: 'Balagruh', required: true },

  category: {
    type: String,
    enum: ['Medical', 'Safety', 'Mental Health', 'Other'],
    required: true
  },
  priority: {
    type: String,
    enum: ['critical', 'high', 'medium'],
    default: function() {
      return this.category === 'Medical' ? 'critical' : 'high';
    }
  },
  description: String,
  location: String, // e.g., "Computer Lab", "Dormitory Room 12"

  status: {
    type: String,
    enum: ['sent', 'acknowledged', 'responding', 'arrived', 'resolved', 'cancelled'],
    default: 'sent'
  },
  responses: [{
    userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
    responseType: {
      type: String,
      enum: ['acknowledged', 'responding', 'arrived']
    },
    timestamp: Date,
    notes: String
  }],
  escalation: {
    tier1SentAt: Date, // Balagruh coaches + in-charge
    tier2SentAt: Date, // Admins + SMS/WhatsApp
    tier3SentAt: Date, // Broadcast all staff
    currentTier: { type: Number, default: 1, min: 1, max: 3 }
  },
  resolution: {
    resolvedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
    resolvedAt: Date,
    resolutionNotes: String,
    outcome: String // "Student received medical attention", etc.
  }
});
```



```

    },

    healthCorrelation: {
      recentHealthRecordId: { type: mongoose.Schema.Types.ObjectId, ref:
'HealthRecord' },
      abnormalVitals: Boolean,
      notes: String
    },

    notifications: [{
      channel: { type: String, enum: ['push', 'sms', 'whatsapp', 'email'] },
      recipientId: mongoose.Schema.Types.ObjectId,
      sentAt: Date,
      delivered: Boolean,
      deliveredAt: Date
    }],

    audit: {
      createdAt: { type: Date, default: Date.now },
      updatedAt: Date,
      createdFrom: { type: String, enum: ['desktop', 'mobile'], default: 'desktop' }
    }
  });

SOSAlertSchema.index({ studentId: 1, 'audit.createdAt': -1 });
SOSAlertSchema.index({ balagruhId: 1, status: 1 });
SOSAlertSchema.index({ status: 1, 'audit.createdAt': -1 });

```

33.3. Message Schema

```

// backend/models/Message.js
const MessageSchema = new mongoose.Schema({
  _id: mongoose.Schema.Types.ObjectId,
  conversationId: { type: mongoose.Schema.Types.ObjectId, ref: 'Conversation',
required: true },
  senderId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },

  type: {
    type: String,
    enum: ['text', 'image', 'video', 'document', 'audio'],
    default: 'text'
  },

  content: {
    text: String, // For text messages
    fileUrl: String, // For media messages (S3 URL)
    fileName: String,
    fileSize: Number, // bytes
    mimeType: String,
    thumbnailUrl: String, // For images/videos
    duration: Number // For audio/video (seconds)
  }
});

```

```

    },

    status: {
      sent: { type: Boolean, default: true },
      delivered: { type: Boolean, default: false },
      read: { type: Boolean, default: false },
      deliveredAt: Date,
      readAt: Date
    },

    replyTo: { type: mongoose.Schema.Types.ObjectId, ref: 'Message' }, // For
message replies

    timestamp: { type: Date, default: Date.now, index: true },
    deletedAt: Date, // Soft delete
    deletedBy: mongoose.Schema.Types.ObjectId
  });

MessageSchema.index({ conversationId: 1, timestamp: -1 });
MessageSchema.index({ senderId: 1, timestamp: -1 });

```

33.4. HealthRecord Schema

```

// backend/models/HealthRecord.js
const HealthRecordSchema = new mongoose.Schema({
  _id: mongoose.Schema.Types.ObjectId,
  studentId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  balagruhId: { type: mongoose.Schema.Types.ObjectId, ref: 'Balagruh', required: true },

  recordType: {
    type: String,
    enum: ['routine', 'incident', 'emergency'],
    required: true
  },

  vitals: {
    weight: { value: Number, unit: 'kg' },
    height: { value: Number, unit: 'cm' },
    temperature: { value: Number, unit: 'celsius' },
    bloodPressure: {
      systolic: Number,
      diastolic: Number
    },
    heartRate: { value: Number, unit: 'bpm' }
  },

  symptoms: [{
    symptom: {
      type: String,

```

```

    enum: ['fever', 'cough', 'headache', 'fatigue', 'nausea', 'dizziness',
'pain', 'other']
  },
  severity: {
    type: String,
    enum: ['mild', 'moderate', 'severe']
  },
  notes: String
}],

incident: {
  description: String,
  actionTaken: String,
  followUpRequired: Boolean,
  photos: [String], // S3 URLs
},

documents: [{
  type: { type: String, enum: ['medical_report', 'prescription', 'lab_result',
'other'] },
  fileUrl: String, // S3 URL
  fileName: String,
  uploadedAt: Date
}],

alerts: [{
  alertType: {
    type: String,
    enum: ['high_temperature', 'weight_change', 'blood_pressure', 'custom']
  },
  message: String,
  severity: {
    type: String,
    enum: ['warning', 'critical']
  },
  triggeredAt: Date,
  acknowledgedBy: mongoose.Schema.Types.ObjectId,
  acknowledgedAt: Date
}],

sosCorrelation: {
  sosAlertId: { type: mongoose.Schema.Types.ObjectId, ref: 'SOSAlert' },
  linkedAt: Date,
  notes: String
},

recordedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true
},
recordedAt: { type: Date, default: Date.now, index: true },
updatedAt: Date
});

HealthRecordSchema.index({ studentId: 1, recordedAt: -1 });

```

```
HealthRecordSchema.index({ balagruhId: 1, recordType: 1 });
HealthRecordSchema.index({ 'alerts.alertType': 1, 'alerts.acknowledgedAt': 1 });
```

33.5. AttendanceUpload Schema

```
// backend/models/AttendanceUpload.js
const AttendanceUploadSchema = new mongoose.Schema({
  _id: mongoose.Schema.Types.ObjectId,
  balagruhId: { type: mongoose.Schema.Types.ObjectId, ref: 'Balagruh', required:
true },
  uploadedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true
},

  photo: {
    originalUrl: String, // S3 URL
    processedUrl: String, // S3 URL (compressed)
    thumbnailUrl: String, // S3 URL
    fileSize: Number, // bytes
    dimensions: {
      width: Number,
      height: Number
    }
  },

  processingStatus: {
    type: String,
    enum: ['queued', 'processing', 'completed', 'failed'],
    default: 'queued'
  },

  recognitionResults: [{
    studentId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
    confidence: Number, // 0-1
    faceBox: {
      x: Number,
      y: Number,
      width: Number,
      height: Number
    },
    status: {
      type: String,
      enum: ['present', 'uncertain', 'manual_override']
    }
  }
],

  metadata: {
    totalFacesDetected: Number,
    totalStudentsRecognized: Number,
    processingTimeMs: Number,
    uploadedFrom: { type: String, enum: ['mobile', 'desktop'], default: 'mobile'
  },
},
```

```

    deviceInfo: {
      platform: String,
      appVersion: String
    }
  },

  attendanceDate: { type: Date, required: true, index: true },
  uploadedAt: { type: Date, default: Date.now },
  processedAt: Date
});

AttendanceUploadSchema.index({ balagruhId: 1, attendanceDate: -1 });
AttendanceUploadSchema.index({ uploadedBy: 1, uploadedAt: -1 });
AttendanceUploadSchema.index({ processingStatus: 1 });

```

34. Appendix C: API Documentation Structure

34.1. Sprint 3 - Mobile APIs

```

/api/v1/mobile:

/auth:
  POST /login:
    Description: Mobile authentication
    Request: { username, password }
    Response: { token, refreshToken, user }

  POST /refresh:
    Description: Refresh access token
    Request: { refreshToken }
    Response: { token }

  POST /logout:
    Description: Invalidate tokens
    Headers: Authorization: Bearer <token>
    Response: { success: true }

/dashboard:
  GET /:userId:
    Description: Get mobile dashboard data
    Response: { quickActions, recentActivity, stats }

/attendance:
  POST /upload-photo:
    Description: Upload attendance photo
    Request: multipart/form-data { photo, balagruhId, date }
    Response: { uploadId, status: 'queued' }

  GET /results/:uploadId:
    Description: Get recognition results

```

```
Response: { recognized: [...], unrecognized: [...] }

GET /history:
  Description: Get attendance history
  Query: { balagruhId, startDate, endDate }
  Response: { uploads: [...] }

/media:
  POST /generate-upload-url:
    Description: Generate S3 presigned URL
    Request: { fileName, contentType, folder }
    Response: { uploadUrl, key, cdnUrl }

  POST /publish:
    Description: Associate uploaded content with course
    Request: { key, courseId, moduleId, metadata }
    Response: { success: true, contentId }

/analytics:
  GET /dashboard:
    Description: Analytics dashboard
    Query: { balagruhId, startDate, endDate }
    Response: { metrics, charts }

  GET /export:
    Description: Export report
    Query: { format: 'pdf' | 'csv', reportType, dateRange }
    Response: { downloadUrl }

/notifications:
  POST /register-device:
    Description: Register FCM token
    Request: { fcmToken, platform, deviceId }
    Response: { success: true }

  GET /history:
    Description: Get notification history
    Query: { limit, offset }
    Response: { notifications: [...], total }

  PUT /mark-read:
    Description: Mark notifications as read
    Request: { notificationIds: [...] }
    Response: { success: true }
```

34.2. Sprint 4 - Emergency & Communication APIs

```
/api/v1/sos:

  POST /trigger:
    Description: Create SOS alert (desktop)
```

```
Request: { studentId, category, description, location }
Response: { sosAlertId, status: 'sent' }
```

POST /respond:

```
Description: Respond to SOS alert (mobile)
Request: { sosAlertId, responseType, notes }
Response: { success: true, status: 'acknowledged' }
```

PUT /:sosAlertId/resolve:

```
Description: Mark SOS as resolved
Request: { resolutionNotes, outcome }
Response: { success: true }
```

GET /active:

```
Description: Get active SOS alerts
Query: { balagruhId }
Response: { alerts: [...] }
```

GET /history:

```
Description: Get SOS alert history
Query: { studentId, balagruhId, startDate, endDate }
Response: { alerts: [...] }
```

/api/v1/messaging:

GET /conversations:

```
Description: Get user's conversations
Response: { conversations: [...] }
```

POST /conversations:

```
Description: Create new conversation
Request: { participants: [...], type: '1-on-1' | 'group', name }
Response: { conversationId }
```

GET /conversations/:conversationId/messages:

```
Description: Get messages in conversation
Query: { limit, before }
Response: { messages: [...], hasMore }
```

POST /conversations/:conversationId/messages:

```
Description: Send message
Request: { text, attachments }
Response: { message }
```

PUT /messages/:messageId/read:

```
Description: Mark message as read
Response: { success: true }
```

POST /upload-attachment:

```
Description: Generate presigned URL for attachment
Request: { fileName, contentType, conversationId }
Response: { uploadUrl, key }
```

/api/v1/health:

```

POST /records:
  Description: Create health record
  Request: { studentId, recordType, vitals, symptoms, incident }
  Response: { healthRecordId }

GET /records/:studentId:
  Description: Get student health records
  Query: { startDate, endDate, recordType }
  Response: { records: [...] }

GET /records/:recordId:
  Description: Get specific health record
  Response: { record }

POST /records/:recordId/documents:
  Description: Upload health document
  Request: multipart/form-data { file, documentType }
  Response: { documentUrl }

GET /alerts:
  Description: Get unacknowledged health alerts
  Query: { balagruhId }
  Response: { alerts: [...] }

PUT /alerts/:alertId/acknowledge:
  Description: Acknowledge health alert
  Response: { success: true }

/api/v1/whatsapp:

POST /send:
  Description: Send WhatsApp message (internal use)
  Request: { templateId, recipientPhone, parameters }
  Response: { messageId, status }

GET /delivery-status/:messageId:
  Description: Check WhatsApp message status
  Response: { status: 'sent' | 'delivered' | 'read' | 'failed' }

```

35. Sign-off Section

35.1. Stakeholder Agreement

This Master Project Specification Document (MPSD) for the combined Sprint 3 and Sprint 4 represents the complete, agreed-upon scope of work for the 28-day development effort. All stakeholders acknowledge that:

1. This document is the single source of truth for all development activities
2. The 28-day timeline is achievable with the defined team structure and resources
3. Any scope changes must go through formal change request process
4. Success criteria are clearly defined and measurable

- 5. Resource allocation (team, infrastructure, devices) has been agreed upon
- 6. Risk mitigation strategies are acceptable

35.2. Approval Signatures

Client (ISF Representative):

- Name: _____
- Title: _____
- Signature: _____
- Date: _____

Project Manager:

- Name: _____
- Signature: _____
- Date: _____

Technical Lead:

- Name: _____
- Signature: _____
- Date: _____

QA Lead:

- Name: _____
- Signature: _____
- Date: _____

Product Owner:

- Name: _____
- Signature: _____
- Date: _____

35.3. Document Control

Version	Date	Author	Changes
1.0	Oct 17, 2025	Anjai Jacob	Initial Sprint 3+4 Combined MPSD
2.0	Oct 17, 2025	Anjai Jacob	Client-facing version (Sprint 2-5 structure)

35.4. Distribution List

- ISF Project Sponsor
- ISF Program Director
- Development Team (7 members)
- QA Team (1 member)
- DevOps Team (1 member)
- External Stakeholders (as required)

36. Post-Implementation Considerations

36.1. Future Sprint Dependencies

Features developed in this combined sprint will enable:

- **Sprint 6:** Video-based liveness detection for enhanced facial recognition
- **Future Sprints:** Automated phone calls for critical SOS alerts
- **Future Sprints:** Multi-language support for mobile UI (Hindi, Marathi)
- **Future Sprints:** Advanced analytics with ML predictions

36.2. Maintenance & Support Plan

- **Week 29-33:** Stabilization period and bug fixes (P2/P3 bugs)
- **Week 34-38:** Performance optimization based on production metrics
- **Week 39+:** Feature enhancements based on user feedback

Service Level Agreement (SLA):

- **P0 (Critical - SOS system down):** < 15 min response, < 2 hour resolution
- **P1 (High - Login broken):** < 1 hour response, < 4 hour resolution
- **P2 (Medium - Feature bug):** < 4 hour response, < 24 hour resolution
- **P3 (Low - Minor issue):** < 24 hour response, < 7 day resolution

36.3. Training Requirements

- **Admin Training:** 1-day workshop on mobile app management and SOS monitoring
- **Coach Training:** Half-day session on mobile features (attendance, media, messaging, SOS response)
- **Balagruh In-Charge Training:** Half-day session on attendance marking and health tracking
- **Documentation:** Video tutorials for all major mobile workflows
- **Support Hotline:** Dedicated support for first 2 weeks post-launch

36.4. Monitoring & KPIs

Key Performance Indicators:

- **Mobile App:**
 - Daily Active Users (DAU): Target 80% of staff
 - App crash rate: < 0.1%
 - Average session duration: > 10 minutes
 - Feature adoption rate: > 70% within 2 weeks
- **SOS System:**
 - Average response time: < 3 minutes (target)
 - Escalation rate: < 20% (most resolved in Tier 1)
 - Resolution rate: 100% (all SOS alerts must be resolved)
- **Attendance:**

- Facial recognition accuracy: > 95%
- Attendance marking time: < 60 seconds per class
- Manual override rate: < 10%

- **Messaging:**

- Message delivery rate: > 98%
- Average delivery time: < 2 seconds
- User adoption: > 80% of staff actively using

END OF DOCUMENT

Total Pages: ~150 **Word Count:** ~27,000 **Last Updated:** October 17, 2025 **Document Type:** Client-Facing
Master Project Specification Document **Classification:** For Client Review and Approval